```
// G17Achievements.cs
using System;
using System.Collections.Generic;
namespace Sy.Unbox
{
    /// &lt;summary&gt;One achievement definition of the lightweight eight (task
    /// S2-T003, requirements doc sec2): stable id plus the i18n name key
    /// (red line 4 - the wording lives in cfg_g17_i18n.json only).&lt;/summary&gt;
    public sealed class G17AchievementDef
    {
        public string Id;
        public string NameKey;
    }
    /// &lt;summary&gt;Toast beat of a freshly unlocked achievement: the view
    /// renders the toast from the i18n face; the beat is the whole payload
    /// contract (kind + id + name key).&lt;/summary&gt;
    public sealed class G17AchievementToast
    {
        public const string KindOfBeat = "achv_toast";
        public string Kind = KindOfBeat;
        public string Id;
        public string NameKey;
    }
    /// &lt;summary&gt;
    /// Lightweight achievement system, eight badges (task S2-T003, G17AlbumPage
    /// paradigm): the rules read sim truths only (RevealCount / RareOrBetter /
    /// CleanCraft / ad-free streak / album set / day streak) - zero UI state,
    /// zero networking (X032: achievements ride local progress only, red line
    /// 12). Evaluate() returns the freshly unlocked toasts once and marks them
    /// unlocked (no re-toast). Thresholds are S2 design estimates recorded
    /// honestly per U099: ad-free clear = 10 consecutive ad-free packages
    /// (one good session - the U091 priority declaration, competitor review
    /// "10 minutes, 8 of it ads" counterpoint), week streak = 7 sim days,
    /// box club = 100 reveals, album half = half of the frozen 24-slot
    /// table. The unlocked-set persistence lands with the S2-T004 save face.
    /// &lt;/summary&gt;
    public sealed class G17Achievements
    {
        public const string FirstUnbox = "first_unbox";
        public const string FirstRare = "first_rare";
        public const string FullCraft = "full_craft";
        public const string AdFreeClear = "ad_free_clear";
        public const string AlbumHalf = "album_half";
        public const string AlbumFull = "album_full";
        public const string WeekStreak = "week_streak";
        public const string BoxClub = "box_club";
        /// &lt;summary&gt;Toast caption i18n key (red line 4).&lt;/summary&gt;
        public const string ToastCaptionKey = "g17.achv.toast";
        // Design-estimate thresholds (numeric pack reconciliation lands with
        // the S2 balance pass; honest defaults, never tuned to force green).
```

```
public const int AdFreeClearRun = 10;
public const int WeekStreakDays = 7;
public const int BoxClubCount = 100;
static readonly G17AchievementDef[] Defs =
{
    new G17AchievementDef { Id = FirstUnbox, NameKey = "g17.achv.first_unbox" },
    new G17AchievementDef { Id = FirstRare, NameKey = "g17.achv.first_rare" },
    new G17AchievementDef { Id = FullCraft, NameKey = "g17.achv.full_craft" },
    new G17AchievementDef { Id = AdFreeClear, NameKey = "g17.achv.ad_free" },
    new G17AchievementDef { Id = AlbumHalf, NameKey = "g17.achv.album_half" },
    new G17AchievementDef { Id = AlbumFull, NameKey = "g17.achv.album_full" },
    new G17AchievementDef { Id = WeekStreak, NameKey = "g17.achv.week_streak" },
    new G17AchievementDef { Id = BoxClub, NameKey = "g17.achv.box_club" },
};
readonly UnboxSim _sim;
readonly G17AlbumCatalog _album;
readonly HashSet&lt;string&gt; _unlocked = new HashSet&lt;string&gt;();
public G17Achievements(UnboxSim sim)
    : this(sim, G17AlbumCatalog.Load())
{
}
public G17Achievements(UnboxSim sim, G17AlbumCatalog album)
{
    _sim = sim ?? throw new ArgumentNullException(nameof(sim));
    _album = album;
}
/// &lt;summary&gt;The frozen eight, definition order = toast evaluation order.&lt;/summary&gt;
public static IReadOnlyList&lt;G17AchievementDef&gt; All { get { return Defs; } }
public int TotalCount { get { return Defs.Length; } }
/// &lt;summary&gt;Frozen album table size (falls back to 4 tiers x 6 items
/// when the catalog is absent - G17UnboxPresenter parity).&lt;/summary&gt;
public int AlbumTotal
{
    get
    {
        return _album != null &amp;&amp; _album.Items != null &amp;&amp; _album.Items.Length &gt; 0
            ? _album.Items.Length
            : G17AlbumCatalog.ItemsPerTier * G17BoxCatalog.TierCount;
    }
}
public int UnlockedCount { get { return _unlocked.Count; } }
public bool IsUnlocked(string id)
{
    return _unlocked.Contains(id);
}
/// &lt;summary&gt;Save face (S2-T004): restores the persisted unlocked
/// set. Only ids of the frozen eight are accepted - unknown or
/// tampered ids drop silently (G09 tamper-clamp paradigm); the set
/// only ever grows and restored ids never re-toast (Evaluate is
/// idempotent against the sim truths).&lt;/summary&gt;
```

```
public void RestoreUnlocked(IEnumerable&lt;string&gt; ids)
{
    if (ids == null) return;
    foreach (var id in ids)
    {
        for (int i = 0; i &lt; Defs.Length; i++)
        {
            if (Defs[i].Id == id)
            {
                _unlocked.Add(id);
                break;
            }
        }
    }
}
/// &lt;summary&gt;Save face (S2-T004): ordered unlocked-id snapshot for
/// serialization (definition order = stable payload).&lt;/summary&gt;
public string[] SnapshotUnlocked()
{
    var list = new List&lt;string&gt;();
    for (int i = 0; i &lt; Defs.Length; i++)
    {
        if (_unlocked.Contains(Defs[i].Id)) list.Add(Defs[i].Id);
    }
    return list.ToArray();
}
/// &lt;summary&gt;Evaluates every rule against the sim truths and returns
/// the freshly unlocked achievements as toast beats (each fires
/// exactly once; later evaluates stay silent for unlocked ids).&lt;/summary&gt;
public G17AchievementToast[] Evaluate()
{
    var fired = new List&lt;G17AchievementToast&gt;();
    int collected = _sim.AlbumIds.Count;
    int total = AlbumTotal;
    for (int i = 0; i &lt; Defs.Length; i++)
    {
        var def = Defs[i];
        if (_unlocked.Contains(def.Id)) continue;
        bool hit;
        switch (def.Id)
        {
            case FirstUnbox: hit = FirstUnboxRule(_sim.RevealCount); break;
            case FirstRare: hit = FirstRareRule(_sim.RareOrBetterReveals); break;
            case FullCraft: hit = FullCraftRule(_sim.CleanCraftReveals); break;
            case AdFreeClear: hit = AdFreeRule(_sim.LongestAdFreeStreak); break;
            case AlbumHalf: hit = AlbumHalfRule(collected, total); break;
            case AlbumFull: hit = AlbumFullRule(collected, total); break;
            case WeekStreak: hit = WeekStreakRule(_sim.LongestRevealDayStreak); break;
            case BoxClub: hit = BoxClubRule(_sim.RevealCount); break;
            default: hit = false; break;
```

```
                }
                if (!hit) continue;
                _unlocked.Add(def.Id);
                fired.Add(new G17AchievementToast { Id = def.Id, NameKey = def.NameKey });
            }
            return fired.ToArray();
        }
        // ---- pure rule face (machine-checkable, thresholds never drift) ----
        public static bool FirstUnboxRule(int revealCount) { return revealCount &gt;= 1; }
        public static bool FirstRareRule(int rareOrBetterReveals) { return rareOrBetterReveals &gt;= 1; }
        public static bool FullCraftRule(int cleanCraftReveals) { return cleanCraftReveals &gt;= 1; }
        public static bool AdFreeRule(int longestAdFreeStreak) { return longestAdFreeStreak &gt;= AdFreeClearRun; }
        /// &lt;summary&gt;Half of the frozen table (12 of 24); an empty table can
        /// never unlock (rules never fire on absent data).&lt;/summary&gt;
        public static bool AlbumHalfRule(int collected, int total)
        {
            return total &gt; 0 &amp;&amp; collected &gt;= total / 2;
        }
        public static bool AlbumFullRule(int collected, int total)
        {
            return G17AlbumPage.WallUnlockedOf(collected, total);
        }
        public static bool WeekStreakRule(int longestRevealDayStreak)
        {
            return longestRevealDayStreak &gt;= WeekStreakDays;
        }
        public static bool BoxClubRule(int revealCount) { return revealCount &gt;= BoxClubCount; }
    }
}
// G17AlbumCatalog.cs
using System;
using System.Collections.Generic;
using Logger = Sy.Core.Logger;
using UnityEngine;
namespace Sy.Unbox
{
    [Serializable]
    public sealed class AlbumItemData
    {
        public string id;        // "C01".."H06" - mirrors the UnboxSim package ItemId space
        public string tier;      // common / rare / absurd / hidden
        public string stem;      // art stem for the S2 illustration batch (X027 pipeline)
        public string nameKey;   // i18n key into cfg_g17_i18n.json (red line 4)
    }
    [Serializable]
    public sealed class AlbumFile
    {
        public AlbumItemData[] items;
    }
    /// &lt;summary&gt;
```

```
/// Static album catalog for the 24-item absurd collection (task S0-T003,
/// requirements doc sec6 frozen table). Data layer only: four rarity tiers
/// carry six items each and the ids mirror the UnboxSim package ItemId
/// space (rarity code + 01..pool). Unknown ids return null and are never
/// an error (G12GameCatalog paradigm). Validate() enforces the numeric
/// pack sec1 pool reconciliation (6 x 4 = 24) plus structural integrity
/// for the EditMode gate family in G17AlbumCatalogTests.
/// &lt;/summary&gt;
public sealed class G17AlbumCatalog
{
    public const string TierCommon = "common";
    public const string TierRare = "rare";
    public const string TierAbsurd = "absurd";
    public const string TierHidden = "hidden";
    // Numeric pack sec1: album pool per tier = 6/6/6/6 (24-item table).
    public const int ItemsPerTier = 6;
    static readonly string[] TierOrder = { TierCommon, TierRare, TierAbsurd, TierHidden };
    public AlbumItemData[] Items { get; }
    G17AlbumCatalog(AlbumFile file)
    {
        Items = file.items;
    }
    public static G17AlbumCatalog Load()
    {
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_g17_album");
        if (asset == null)
        {
            Logger.Warn("G17.Album", "cfg_g17_album.json not found under Resources");
            return null;
        }
        var file = JsonUtility.FromJson&lt;AlbumFile&gt;(asset.text);
        if (file == null || file.items == null || file.items.Length == 0)
        {
            Logger.Warn("G17.Album", "cfg_g17_album.json failed to parse");
            return null;
        }
        return new G17AlbumCatalog(file);
    }
    /// &lt;summary&gt;Unknown item ids (and empty ids) return null - callers must
    /// treat that as "not configured", never as an error.&lt;/summary&gt;
    public AlbumItemData ItemById(string id)
    {
        if (string.IsNullOrEmpty(id)) return null;
        for (int i = 0; i &lt; Items.Length; i++)
        {
            if (Items[i].id == id) return Items[i];
        }
        return null;
    }
    public int CountTier(string tier)
```

```
        {
            int n = 0;
            for (int i = 0; i &lt; Items.Length; i++)
            {
                if (Items[i].tier == tier) n++;
            }
            return n;
        }
        /// &lt;summary&gt;
        /// Structural validation shared by the EditMode tests: unique ids,
        /// valid tiers, exactly ItemsPerTier per tier (numeric pack sec1 pool
        /// reconciliation 6 x 4), unique stems, non-empty i18n keys. Returns
        /// the problem list; empty means the table is well-formed.
        /// &lt;/summary&gt;
        public List&lt;string&gt; Validate()
        {
            var problems = new List&lt;string&gt;();
            var ids = new HashSet&lt;string&gt;();
            var stems = new HashSet&lt;string&gt;();
            for (int i = 0; i &lt; Items.Length; i++)
            {
                var it = Items[i];
                string tag = "album[" + i + "]";
                if (string.IsNullOrEmpty(it.id)) problems.Add(tag + " missing id");
                else if (!ids.Add(it.id)) problems.Add(tag + " duplicate id " + it.id);
                bool tierOk = it.tier == TierCommon || it.tier == TierRare ||
                              it.tier == TierAbsurd || it.tier == TierHidden;
                if (!tierOk) problems.Add(tag + " tier must be common/rare/absurd/hidden, got " + it.tier);
                if (string.IsNullOrEmpty(it.stem)) problems.Add(tag + " missing stem");
                else if (!stems.Add(it.stem)) problems.Add(tag + " duplicate stem " + it.stem);
                if (string.IsNullOrEmpty(it.nameKey)) problems.Add(tag + " missing nameKey");
            }
            for (int t = 0; t &lt; TierOrder.Length; t++)
            {
                int n = CountTier(TierOrder[t]);
                if (n != ItemsPerTier)
                {
                    problems.Add("tier " + TierOrder[t] + " must hold exactly " + ItemsPerTier +
                                 " items (numeric pack sec1 pool), got " + n);
                }
            }
            return problems;
        }
    }
}
// G17AlbumPage.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.Unbox
```

```
{
    /// &lt;summary&gt;One beat of the collection-page flow (task S2-T002): the open
    /// beat carries the progress snapshot (collected / remaining / wall) so
    /// the view can render the anchor in the same frame; the close beat is
    /// the bare kind.&lt;/summary&gt;
    public sealed class G17AlbumPageBeat
    {
        public string Kind;        // "album_open" | "album_close"
        public int Collected;      // unique collected count at beat time (open beat)
        public int Remaining;      // items still missing from the frozen table (open beat)
        public bool WallUnlocked;  // full-collection achievement wall state (open beat)
    }
    /// &lt;summary&gt;One grid slot of the collection page: item id, rarity tier,
    /// collected state, four-tier color mark and the i18n name key.&lt;/summary&gt;
    public sealed class G17AlbumCell
    {
        public string Id;        // "C01".."H06" (frozen 24-slot table)
        public int Tier;         // 0 common / 1 rare / 2 absurd / 3 hidden
        public bool Collected;   // mirrors the live sim album set
        public Color TierColor;  // four-tier color mark (white/blue/purple/gold)
        public string NameKey;   // i18n key of the item name (red line 4)
    }
    /// &lt;summary&gt;
    /// Pure view-model of the collection overlay page (task S2-T002 system
    /// face, G17UnboxPresenter paradigm): owns the open/close beat flow, builds
    /// the 24-cell grid from the frozen album table with the four-tier color
    /// marks (white common / blue rare / purple absurd / gold hidden), exposes
    /// the collected/uncollected cell states, the "X away from the full set"
    /// progress anchor through the i18n face and the full-collection
    /// achievement wall slot. No MonoBehaviour and no UGUI types: EditMode
    /// tests pin the beats, grid, states and the i18n family. Scene wiring of
    /// the overlay = later slice.
    /// &lt;/summary&gt;
    public sealed class G17AlbumPage
    {
        public const string BeatOpen = "album_open";
        public const string BeatClose = "album_close";
        // Four-tier color marks (requirements doc sec6): white common, blue
        // rare, purple absurd, gold hidden - the four anchors stay distinct
        // and bot-tellable before the S2 art batch lands.
        public static readonly Color ColorCommon = new Color(0.92f, 0.95f, 0.98f);
        public static readonly Color ColorRare = new Color(0.24f, 0.55f, 1.00f);
        public static readonly Color ColorAbsurd = new Color(0.62f, 0.34f, 0.96f);
        public static readonly Color ColorHidden = new Color(1.00f, 0.80f, 0.24f);
        public static readonly Color[] TierColors = { ColorCommon, ColorRare, ColorAbsurd, ColorHidden };
        static readonly G17AlbumPageBeat[] NoBeats = new G17AlbumPageBeat[0];
        readonly UnboxSim _sim;
        readonly G17AlbumCatalog _album;
        public bool IsOpen { get; private set; }
        public G17AlbumPage(UnboxSim sim, G17AlbumCatalog album)
```

```
{
    _sim = sim ?? throw new ArgumentNullException(nameof(sim));
    _album = album;
}
/// &lt;summary&gt;Catalog absent = the page cannot build its grid; Open()
/// then stays a guarded no-op (G12 null-safe paradigm, never an error).&lt;/summary&gt;
public bool IsAvailable
{
    get { return _album != null &amp;&amp; _album.Items != null &amp;&amp; _album.Items.Length &gt; 0; }
}
/// &lt;summary&gt;Opens the page: exactly one album_open beat carrying the
/// progress snapshot. Guarded no-ops hand back an empty array: opening
/// an already-open page, or opening with the catalog unavailable.&lt;/summary&gt;
public G17AlbumPageBeat[] Open()
{
    if (!IsAvailable || IsOpen) return NoBeats;
    IsOpen = true;
    return new[]
    {
        new G17AlbumPageBeat
        {
            Kind = BeatOpen,
            Collected = CollectedCount,
            Remaining = RemainingToFull,
            WallUnlocked = WallUnlocked,
        }
    };
}
/// &lt;summary&gt;Closes the page: exactly one album_close beat. Closing a
/// closed page is a guarded no-op (empty array).&lt;/summary&gt;
public G17AlbumPageBeat[] Close()
{
    if (!IsOpen) return NoBeats;
    IsOpen = false;
    return new[] { new G17AlbumPageBeat { Kind = BeatClose } };
}
/// &lt;summary&gt;Grid size: the frozen 24-slot table (0 when unavailable).&lt;/summary&gt;
public int TotalCount
{
    get { return IsAvailable ? _album.Items.Length : 0; }
}
/// &lt;summary&gt;Unique collected ids at read time (mirrors the HUD album
/// progress badge numerator).&lt;/summary&gt;
public int CollectedCount
{
    get { return _sim.AlbumIds.Count; }
}
/// &lt;summary&gt;"X away from the full set" progress anchor numerator.&lt;/summary&gt;
public int RemainingToFull
{
```

```
    get { return RemainingOf(CollectedCount, TotalCount); }
}
/// &lt;summary&gt;Full-collection achievement wall slot: lights only when
/// every slot of the frozen table is collected.&lt;/summary&gt;
public bool WallUnlocked
{
    get { return WallUnlockedOf(CollectedCount, TotalCount); }
}
/// &lt;summary&gt;Pure wall rule: unlocked iff the table is non-empty and
/// every slot is collected (collected &amp;gt;= total clamps overfills).&lt;/summary&gt;
public static bool WallUnlockedOf(int collected, int total)
{
    return total &gt; 0 &amp;&amp; collected &gt;= total;
}
/// &lt;summary&gt;Pure remaining rule: missing count of the frozen table
/// (clamped to [0, total]; negatives never leak into the anchor).&lt;/summary&gt;
public static int RemainingOf(int collected, int total)
{
    if (collected &lt; 0) return total &lt; 0 ? 0 : total;
    if (collected &gt; total) return 0;
    return total - collected;
}
/// &lt;summary&gt;Progress-anchor label through the i18n face
/// (g17.view.album.stamp holds the "{0}" slot, red line 4: the zh
/// wording lives in the config file, never in source).&lt;/summary&gt;
public string RemainingText()
{
    return string.Format(G17Text.Get("g17.view.album.stamp"), RemainingToFull);
}
/// &lt;summary&gt;The 24-cell grid in catalog order (tier blocks C/R/A/H,
/// six cells each). Rebuilds on every call so the collected states
/// mirror the live sim; empty array when the catalog is unavailable.&lt;/summary&gt;
public G17AlbumCell[] Cells()
{
    if (!IsAvailable) return new G17AlbumCell[0];
    var items = _album.Items;
    var collected = _sim.AlbumIds;
    var cells = new G17AlbumCell[items.Length];
    for (int i = 0; i &lt; items.Length; i++)
    {
        var it = items[i];
        int tier = TierIndexOf(it.tier);
        cells[i] = new G17AlbumCell
        {
            Id = it.id,
            Tier = tier,
            Collected = collected.Contains(it.id),
            TierColor = TierColors[tier],
            NameKey = it.nameKey,
        };
```

```
            }
            return cells;
        }
        /// &lt;summary&gt;Tier name key of the color legend (i18n face, red line 4).&lt;/summary&gt;
        public static string TierNameKey(int tier)
        {
            switch (tier)
            {
                case 0: return "g17.album.tier.common";
                case 1: return "g17.album.tier.rare";
                case 2: return "g17.album.tier.absurd";
                case 3: return "g17.album.tier.hidden";
                default: return "g17.album.tier.common";
            }
        }
        /// &lt;summary&gt;Tier index of a catalog tier string (unknown tiers fold to
        /// common - the catalog Validate() gate owns rejecting bad tiers).&lt;/summary&gt;
        public static int TierIndexOf(string tier)
        {
            switch (tier)
            {
                case G17AlbumCatalog.TierCommon: return 0;
                case G17AlbumCatalog.TierRare: return 1;
                case G17AlbumCatalog.TierAbsurd: return 2;
                case G17AlbumCatalog.TierHidden: return 3;
                default: return 0;
            }
        }
    }
}
// G17BoxCatalog.cs
using System;
using System.Collections.Generic;
using Logger = Sy.Core.Logger;
using UnityEngine;
namespace Sy.Unbox
{
    [Serializable]
    public sealed class BoxData
    {
        public int tier;        // 0..3 - order index, must appear exactly once (task S1-T002)
        public string rarity;   // common / rare / absurd / hidden - canonical pairing per tier
        public int weight;      // rarity roll weight, numeric pack sec1: 60/25/11/4, sum=100
        public int bubbleMin;   // Random.Next lower bound (inclusive)
        public int bubbleMax;   // Random.Next upper bound (EXCLUSIVE) - sim Next() semantics
        public int latchMin;
        public int latchMax;    // EXCLUSIVE upper bound, same Next() semantics
        public int reveal;      // reveal show tier 0..3 (white single tone ... gold layered fanfare)
        public string assetSlot; // art slot A02..A05 (requirements doc sec4 first batch table)
        public string nameKey;   // i18n key into cfg_g17_i18n.json (red line 4)
```

```
}
[Serializable]
public sealed class BoxFile
{
    public BoxData[] boxes;
}
/// &lt;summary&gt;
/// Static box-type catalog for the four rarity tiers (task S1-T002, GDD sec8
/// schema). Data layer only: the table mirrors the UnboxSim production
/// constants (numeric pack sec1+sec2) so the reconciliation tests can gate
/// zero drift between config and runtime. Unknown tiers return null and
/// are never an error (G17AlbumCatalog paradigm). Validate() enforces the
/// task-book acceptance set: unique tier order, bubble N within 8-15,
/// latch M within 2-4, rarity weights exactly 60/25/11/4 summing to 100.
/// &lt;/summary&gt;
public sealed class G17BoxCatalog
{
    public const int TierCount = 4;
    // Numeric pack sec1 frozen face (thresholds never loosened).
    public static readonly int[] FrozenWeights = { 60, 25, 11, 4 };
    public static readonly string[] FrozenRarityOrder = { "common", "rare", "absurd", "hidden" };
    public static readonly string[] FrozenAssetSlots = { "A02", "A03", "A04", "A05" };
    // Numeric pack sec2 craft bounds: achievable bubble N in [8,15],
    // achievable latch M in [2,4] (Next max is exclusive).
    public const int BubbleFloor = 8;
    public const int BubbleCeiling = 15;
    public const int LatchFloor = 2;
    public const int LatchCeiling = 4;
    public BoxData[] Boxes { get; }
    G17BoxCatalog(BoxFile file)
    {
        Boxes = file.boxes;
    }
    public static G17BoxCatalog Load()
    {
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_g17_boxes");
        if (asset == null)
        {
            Logger.Warn("G17.Box", "cfg_g17_boxes.json not found under Resources");
            return null;
        }
        var file = JsonUtility.FromJson&lt;BoxFile&gt;(asset.text);
        if (file == null || file.boxes == null || file.boxes.Length == 0)
        {
            Logger.Warn("G17.Box", "cfg_g17_boxes.json failed to parse");
            return null;
        }
        return new G17BoxCatalog(file);
    }
    /// &lt;summary&gt;Unknown tiers (or out-of-range) return null - callers must
```

```
/// treat that as "not configured", never as an error.&lt;/summary&gt;
public BoxData BoxByTier(int tier)
{
    if (tier &lt; 0) return null;
    for (int i = 0; i &lt; Boxes.Length; i++)
    {
        if (Boxes[i].tier == tier) return Boxes[i];
    }
    return null;
}
/// &lt;summary&gt;
/// Structural validation for the EditMode gate family: exactly four
/// tiers 0..3 each once (unique tier order), canonical rarity pairing,
/// frozen weights 60/25/11/4 summing to 100, craft bounds keeping the
/// achievable bubble N inside 8-15 and latch M inside 2-4, reveal show
/// tier aligned with rarity tier, unique asset slots, non-empty i18n
/// keys. Returns the problem list; empty means the table is well-formed.
/// &lt;/summary&gt;
public List&lt;string&gt; Validate()
{
    var problems = new List&lt;string&gt;();
    if (Boxes.Length != TierCount)
    {
        problems.Add("box table must hold exactly " + TierCount + " tiers, got " + Boxes.Length);
    }
    var slots = new HashSet&lt;string&gt;();
    var keys = new HashSet&lt;string&gt;();
    int weightSum = 0;
    for (int t = 0; t &lt; TierCount; t++)
    {
        var box = BoxByTier(t);
        string tag = "box[tier " + t + "]";
        if (box == null)
        {
            problems.Add(tag + " missing - tier order must be unique 0.." + (TierCount - 1));
            continue;
        }
        if (box.rarity != FrozenRarityOrder[t])
        {
            problems.Add(tag + " rarity must be " + FrozenRarityOrder[t] + " (numeric pack sec1 pairing), got "
        }
        if (box.weight != FrozenWeights[t])
        {
            problems.Add(tag + " weight must be " + FrozenWeights[t] + " (numeric pack sec1), got " + box.weight
        }
        weightSum += box.weight;
        if (box.bubbleMin &lt; BubbleFloor || box.bubbleMax - 1 &gt; BubbleCeiling || box.bubbleMin &gt;= box.bu
        {
            problems.Add(tag + " bubble bounds must keep N inside " + BubbleFloor + "-" + BubbleCeiling +
                         " (Next exclusive max), got [" + box.bubbleMin + "," + box.bubbleMax + ")");
```

```
                }
                if (box.latchMin &lt; LatchFloor || box.latchMax - 1 &gt; LatchCeiling || box.latchMin &gt;= box.latchMa
                {
                    problems.Add(tag + " latch bounds must keep M inside " + LatchFloor + "-" + LatchCeiling +
                                 " (Next exclusive max), got [" + box.latchMin + "," + box.latchMax + ")");
                }
                if (box.reveal != t)
                {
                    problems.Add(tag + " reveal show tier must equal rarity tier, got " + box.reveal);
                }
                if (string.IsNullOrEmpty(box.assetSlot))
                {
                    problems.Add(tag + " missing assetSlot");
                }
                else if (box.assetSlot != FrozenAssetSlots[t])
                {
                    problems.Add(tag + " assetSlot must be " + FrozenAssetSlots[t] + " (requirements doc sec4), got " +
                }
                else if (!slots.Add(box.assetSlot))
                {
                    problems.Add(tag + " duplicate assetSlot " + box.assetSlot);
                }
                if (string.IsNullOrEmpty(box.nameKey))
                {
                    problems.Add(tag + " missing nameKey");
                }
                else if (!keys.Add(box.nameKey))
                {
                    problems.Add(tag + " duplicate nameKey " + box.nameKey);
                }
            }
            if (weightSum != 100)
            {
                problems.Add("rarity weights must sum to 100 (numeric pack sec1), got " + weightSum);
            }
            return problems;
        }
    }
}
// G17DoublesFunnel.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
using Sy.Telemetry;
namespace Sy.Unbox
{
    /// &lt;summary&gt;
    /// Double-open ad funnel for G17 (task S2-T005, the G12AdFunnel
    /// three-piece paradigm, Mock face): (1) CanOffer pre-checks the point's
```

```
/// category budget on the SAME policy instance the suite's ads service
/// consumes from - an over-cap offer never fires, no dead ad buttons;
/// (2) the accept flow goes through the category-aware ShowRewarded so
/// the daily caps read exactly as designed (double 5/day inside the
/// global 12); (3) every transition lands as one of the evt_double_open
/// five sub-ids (offer/show/complete/landed/capped) so the funnel is
/// auditable end to end. The per-package gate law (numeric pack sec4:
/// 1 per package, consumed by the granted request only) lives sim-side
/// in UnboxSim - this class owns the daily/ad face only. Decline is the
/// U091 zero-harm face: a declined offer is just the normal single
/// reveal, no funnel call, no penalty.
/// &lt;/summary&gt;
public sealed class G17DoublesFunnel
{
    public const string PointId = "g17_doubles";
    readonly TelemetryService _telemetry;
    readonly IAdsService _ads;
    readonly AdCapPolicy _policy;
    readonly Func&lt;string&gt; _dayKey;
    public G17DoublesFunnel(TelemetryService telemetry, IAdsService ads, AdCapPolicy policy, Func&lt;string&gt; dayK
    {
        _telemetry = telemetry;
        _ads = ads;
        _policy = policy;
        _dayKey = dayKey;
    }
    /// &lt;summary&gt;Pre-offer gate: true when the double-open category still
    /// has budget today (uncategorized points always pass). Null policy
    /// face = legacy ungated behavior, never a silent block.&lt;/summary&gt;
    public bool CanOffer()
    {
        if (_policy == null) return true;
        string key = _dayKey != null ? _dayKey() : AdCapPolicy.TodayKey();
        return _policy.HasCategoryBudget(key, AdCategories.CategoryOfPoint(PointId));
    }
    /// &lt;summary&gt;Accept flow entry: books evt_double_open_show, then shows
    /// the rewarded view through the category-aware service. Callers await
    /// the returned task and report complete/landed from the result - an
    /// ad reward is never granted silently and never lost silently.&lt;/summary&gt;
    public Task&lt;RewardResult&gt; Accept(string reason)
    {
        Track(UnboxEventIds.DoubleOpenShow, reason);
        return _ads.ShowRewarded(PointId);
    }
    public void ReportOffer(string reason) =&gt; Track(UnboxEventIds.DoubleOpenOffer, reason);
    public void ReportCapped(string reason) =&gt; Track(UnboxEventIds.DoubleOpenCapped, reason);
    public void ReportComplete(string reason) =&gt; Track(UnboxEventIds.DoubleOpenComplete, reason);
    public void ReportLanded(string reason) =&gt; Track(UnboxEventIds.DoubleOpenLanded, reason);
    void Track(string evtId, string reason)
    {
```

```
            _telemetry?.Track(evtId, new Dictionary&lt;string, string&gt;
            {
                { "point", PointId },
                { "reason", reason ?? "" },
                { "category", AdCategories.CategoryOfPoint(PointId) },
            });
        }
    }
}
// G17HookFlow.cs
using UnityEngine;
namespace Sy.Unbox
{
    /// &lt;summary&gt;
    /// Ten-second hook staging table (task S1-T006, red line 11: docs/05 v0 is
    /// the threshold source; G12HookFlow paradigm). The beat truths already
    /// live in the frozen sim (caption 0.00s = H1, click-fallback seal cut
    /// &lt;= 3s = H2, first bubble pop &lt;= 1s after the cut = H3, three craft
    /// actions inside 10s = H4, first forced-Rare reveal = H5); this table
    /// stages the visible drama of the T+0-3s hook picture on top of those
    /// frozen beats: the vermilion seal pulses brighter from t=0 so the first
    /// operation's affordance is lit the same frame the caption lands, the
    /// seal cut lands as a decaying slash punch that throws the table, and the
    /// first rare reveal lands as a scale punch on the reveal card - the first
    /// wow reads as ONE beat with its fanfare. Pure constants + pure math over
    /// frozen sim clocks only - the sim itself is never touched, so EditMode
    /// tests pin the whole hook timeline without a scene.
    /// &lt;/summary&gt;
    public static class G17HookFlow
    {
        // T+0-3s hook window (docs/05: the hook picture must read inside 3s).
        public const float HookWindowSec = 3.0f;
        // seal glow pulse (H1/H2 affordance): the strip breathes brighter on a
        // slow cadence, lit from the very first frame (visual signature
        // same-frame law, docs/05 T+0 row) - a beckoning affordance, never a
        // strobe (the cadence stays well under seizure-face territory).
        public const float SealGlowCadenceHz = 1.6f;
        public const int SealGlowHex = 0xFF8A6A;   // brighter vermilion pulse face
        /// &lt;summary&gt;Glow level 0..1 for a sim clock (same input, same level).
        /// Starts mid-bright at t=0 and rises, so the strip reads lit on the
        /// caption frame and keeps breathing through the hook window.&lt;/summary&gt;
        public static float SealGlowLevel(float timeSec)
        {
            if (timeSec &lt; 0f) return 0.5f;
            return 0.5f + 0.5f * Mathf.Sin(Mathf.PI * 2f * SealGlowCadenceHz * timeSec);
        }
        // seal-cut slash punch (H3 open): the cut throws the whole table a
        // little - a short decaying canvas offset whose window stays well
        // inside the first-pop beat (H3 &lt;= 1s budget) so the slash and the
        // first pop read as one continuous thrill, never two.
```

```
        public const float SlashPunchDurationSec = 0.18f;
        public const float SlashPunchAmpPx = 16f;   // canvas units on the 1080x1920 board
        public const float SlashPunchYScale = 0.6f; // vertical share: asymmetry reads organic
        /// &lt;summary&gt;Deterministic slash offset for a remaining window (same
        /// input, same offset). The offset fades linearly to rest at the
        /// window edge - the punch decays, never cuts.&lt;/summary&gt;
        public static Vector2 SlashPunchOffset(float windowSec)
        {
            if (windowSec &lt;= 0f || windowSec &gt; SlashPunchDurationSec) return Vector2.zero;
            float t = SlashPunchDurationSec - windowSec;   // window counts down; phase runs forward
            const float w = Mathf.PI * 2f * 13f;
            float fade = windowSec / SlashPunchDurationSec;
            return new Vector2(
                SlashPunchAmpPx * Mathf.Sin(w * t) * fade,
                SlashPunchAmpPx * SlashPunchYScale * Mathf.Cos(w * 1.31f * t) * fade);
        }
        // reveal wow punch (H5): the first rare reveal card pops with a short
        // decaying scale oscillation - the card swells and settles around its
        // rest scale, so the wow beat lands as ONE pulse with its fanfare.
        public const float WowPunchDurationSec = 0.30f;
        public const float WowPunchScaleAmp = 0.06f;   // +/- 6% around rest scale
        /// &lt;summary&gt;Deterministic reveal-card scale for a remaining window
        /// (same input, same scale). Oscillates around 1 and settles to
        /// exactly 1.0 at the window edge - the card decays to rest, never
        /// snaps.&lt;/summary&gt;
        public static float WowPunchScale(float windowSec)
        {
            if (windowSec &lt;= 0f || windowSec &gt; WowPunchDurationSec) return 1f;
            float t = WowPunchDurationSec - windowSec;
            const float w = Mathf.PI * 2f * 11f;
            float fade = windowSec / WowPunchDurationSec;
            return 1f + WowPunchScaleAmp * Mathf.Sin(w * t) * fade;
        }
    }
}
// G17RescueFunnel.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
using Sy.Telemetry;
namespace Sy.Unbox
{
    /// &lt;summary&gt;
    /// Rescue-seat ad funnel for G17 (task S1-T005, the G12AdFunnel
    /// three-piece paradigm, Mock face in S1): (1) CanOffer pre-checks the
    /// point's category budget on the SAME policy instance the suite's ads
    /// service consumes from - an over-cap offer never fires, no dead ad
    /// buttons; (2) the accept flow goes through the category-aware
    /// ShowRewarded so the daily caps read exactly as designed (rescue
```

```
/// 4/day inside the global 12); (3) every transition lands as one of
/// the evt_rescue five sub-ids (offer/show/complete/landed/capped) so
/// the funnel is auditable end to end. The per-box seat law (numeric
/// pack sec3: 1 per box, consumed by the first offer) lives sim-side in
/// UnboxSim - this class owns the daily/ad face only.
/// &lt;/summary&gt;
public sealed class G17RescueFunnel
{
    public const string PointId = "g17_rescue";
    readonly TelemetryService _telemetry;
    readonly IAdsService _ads;
    readonly AdCapPolicy _policy;
    readonly Func&lt;string&gt; _dayKey;
    public G17RescueFunnel(TelemetryService telemetry, IAdsService ads, AdCapPolicy policy, Func&lt;string&gt; dayKe
    {
        _telemetry = telemetry;
        _ads = ads;
        _policy = policy;
        _dayKey = dayKey;
    }
    /// &lt;summary&gt;Pre-offer gate: true when the rescue category still has
    /// budget today (uncategorized points always pass). Null policy face =
    /// legacy ungated behavior, never a silent block.&lt;/summary&gt;
    public bool CanOffer()
    {
        if (_policy == null) return true;
        string key = _dayKey != null ? _dayKey() : AdCapPolicy.TodayKey();
        return _policy.HasCategoryBudget(key, AdCategories.CategoryOfPoint(PointId));
    }
    /// &lt;summary&gt;Accept flow entry: books evt_rescue_show, then shows the
    /// rewarded view through the category-aware service. Callers await the
    /// returned task and report complete/landed from the result - an ad
    /// reward is never granted silently and never lost silently.&lt;/summary&gt;
    public Task&lt;RewardResult&gt; Accept(string reason)
    {
        Track(UnboxEventIds.RescueShow, reason);
        return _ads.ShowRewarded(PointId);
    }
    public void ReportOffer(string reason) =&gt; Track(UnboxEventIds.RescueOffer, reason);
    public void ReportCapped(string reason) =&gt; Track(UnboxEventIds.RescueCapped, reason);
    public void ReportComplete(string reason) =&gt; Track(UnboxEventIds.RescueComplete, reason);
    public void ReportLanded(string reason) =&gt; Track(UnboxEventIds.RescueLanded, reason);
    void Track(string evtId, string reason)
    {
        _telemetry?.Track(evtId, new Dictionary&lt;string, string&gt;
        {
            { "point", PointId },
            { "reason", reason ?? "" },
            { "category", AdCategories.CategoryOfPoint(PointId) },
        });
```

```
        }
    }
}
// G17RevisitPopup.cs
namespace Sy.Unbox
{
    /// &lt;summary&gt;One welcome-back popup beat (task S2-T006, G17AlbumPageBeat
    /// paradigm): Open carries the arrived-box count, the close beat mirrors
    /// the count so the summary can restage if it ever replays.&lt;/summary&gt;
    public struct G17RevisitBeat
    {
        public bool Open;
        public int Arrived;
    }
    /// &lt;summary&gt;
    /// M08 welcome-back popup (task S2-T006, requirements doc sec2): a pure
    /// view model for the offline summary layer - "N boxes arrived while you
    /// were away", dismissed by the first unbox beat (unboxing IS the revisit
    /// hook, never a punishment: no forced break, no progress wipe). Single
    /// open/close beats with empty guards mirror the album-page law. The
    /// cap law is the numeric pack sec5 table read as a total cap: the first
    /// offline day accrues 2 boxes, two or more days accrue 5 (design
    /// estimate - a cap, not a per-day salary).
    /// &lt;/summary&gt;
    public sealed class G17RevisitPopup
    {
        public const int FirstDayCap = 2;
        public const int LaterDaysCap = 5;
        public bool IsOpen { get; private set; }
        public int ArrivedBoxes { get; private set; }
        bool _openedOnce;
        /// &lt;summary&gt;Offline-arrival cap law (numeric pack sec5): zero gap =
        /// no return, one day away = 2 boxes, two or more days = 5 boxes.&lt;/summary&gt;
        public static int ArrivalsForOfflineDays(int offlineDays)
        {
            if (offlineDays &lt;= 0) return 0;
            return offlineDays == 1 ? FirstDayCap : LaterDaysCap;
        }
        /// &lt;summary&gt;Opens the popup for the given arrived count. Guards: a
        /// non-positive count, a second open while already open, or any open
        /// after the one popup of this visit has fired produce an empty beat
        /// batch (single-open law: once per presenter lifetime, unlike the
        /// user-toggled album page - the welcome-back popup never replays).&lt;/summary&gt;
        public G17RevisitBeat[] Open(int arrivedBoxes)
        {
            if (arrivedBoxes &lt;= 0 || IsOpen || _openedOnce) return EmptyBeats;
            IsOpen = true;
            _openedOnce = true;
            ArrivedBoxes = arrivedBoxes;
            return new[] { new G17RevisitBeat { Open = true, Arrived = arrivedBoxes } };
```

```
        }
        /// &lt;summary&gt;M08 "unboxing is the revisit hook": the first craft beat
        /// closes the summary. A close while already closed is an empty
        /// batch (guard law).&lt;/summary&gt;
        public G17RevisitBeat[] CloseOnCraft()
        {
            if (!IsOpen) return EmptyBeats;
            IsOpen = false;
            return new[] { new G17RevisitBeat { Open = false, Arrived = ArrivedBoxes } };
        }
        static readonly G17RevisitBeat[] EmptyBeats = new G17RevisitBeat[0];
    }
}
// G17Save.cs
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.Unbox
{
    /// &lt;summary&gt;Outcome of a save-slot load (task S2-T004, G12/G06 four-state paradigm).&lt;/summary&gt;
    public enum G17LoadStatus
    {
        NewGame,         // slot empty: fresh profile, nothing written yet
        Loaded,          // current-version payload applied as-is
        Migrated,        // older payload upgraded in place (defaults for missing fields)
        CorruptedReset   // unreadable or future-version payload: profile reset to fresh
    }
    /// &lt;summary&gt;
    /// Persistence profile for G17 (task S2-T004): album collection ids,
    /// the honor-face coin ledger, the unlocked achievement set, and the
    /// production-calendar streak chain (seven-day sign-in) plus the offline
    /// timestamp that anchors it. Local storage is the only authority (red
    /// line 12 zero-server doctrine): the platform storage adapter is the
    /// Sy.Core save service, so minigame storage and editor storage ride
    /// one interface with zero game-side branching.
    /// &lt;/summary&gt;
    public sealed class G17Profile
    {
        public List&lt;string&gt; albumIds = new List&lt;string&gt;();
        public int coins;
        public List&lt;string&gt; achievementsUnlocked = new List&lt;string&gt;();
        public int revealDayStreak;
        public int longestRevealDayStreak;
        public long lastRealDayUnix;
    }
    /// &lt;summary&gt;
    /// Single-key JSON persistence through the Sy.Core save service (task
    /// S2-T004, G12SaveSystem paradigm). Four load states mirror the family:
    /// empty slot = NewGame, versionless payload = Migrated with defaults,
    /// garbage or future version = CorruptedReset (a save can brick a run
```

```
/// only by refusing to load, never by throwing). Tamper clamps: coins
/// and streaks clamp at zero, album ids must survive the frozen catalog
/// table, achievement ids must belong to the frozen eight (G09 tamper
/// paradigm). The production calendar face maps the offline timestamp
/// to real day indexes so the seven-day streak survives sessions:
/// same-day return keeps the streak (no stacking), next-day return
/// extends it on the first reveal, a skipped day starts fresh.
/// &lt;/summary&gt;
public static class G17SaveSystem
{
    public const string Slot = "g17_main";
    public const string VersionKey = "g17v";
    public const int CurrentVersion = 1;
    public const long RealDaySeconds = 86400;
    // ---------------- production calendar face (offline timestamp) ----------------
    /// &lt;summary&gt;Real calendar day index of a unix timestamp (clamp at zero).&lt;/summary&gt;
    public static int RealDayIndexOf(long unixSeconds)
    {
        return unixSeconds &lt;= 0 ? 0 : (int)(unixSeconds / RealDaySeconds);
    }
    /// &lt;summary&gt;Streak boot policy: returning today or tomorrow keeps the
    /// chain; skipping one or more days resets it (a negative gap from a
    /// tampered clock also resets - any deterministic policy is honest).&lt;/summary&gt;
    public static int StreakAfterGap(int savedStreak, int gapDays)
    {
        if (savedStreak &lt; 0) savedStreak = 0;
        return gapDays == 0 || gapDays == 1 ? savedStreak : 0;
    }
    /// &lt;summary&gt;Maps the saved streak onto the boot day (task S2-T004
    /// production-calendar closure). Outcomes feed UnboxSim.
    /// SeedRevealDayStreak directly: lastDayIndex = today means today is
    /// already on record (same-day return never stacks), yesterday means
    /// the first reveal of today extends the chain, -1 means fresh.&lt;/summary&gt;
    public static void StreakSeedForBoot(G17Profile profile, long nowUnix, out int currentStreak, out int lastDayInd
    {
        if (profile == null || profile.lastRealDayUnix &lt;= 0)
        {
            currentStreak = 0;
            lastDayIndex = -1;
            return;
        }
        int savedDay = RealDayIndexOf(profile.lastRealDayUnix);
        int today = RealDayIndexOf(nowUnix);
        int gap = today - savedDay;
        currentStreak = StreakAfterGap(profile.revealDayStreak, gap);
        if (gap == 0) lastDayIndex = today;
        else if (gap == 1) lastDayIndex = today - 1;
        else lastDayIndex = -1;
    }
    // ---------------- capture (live truths -&gt; profile) ----------------
```

```
/// &lt;summary&gt;Fills the profile from the live truths: sim album set and
/// streak counters, achievements unlocked snapshot, honor coin
/// ledger, offline timestamp = now. Null faces degrade to empty
/// values instead of throwing (boot-path safety).&lt;/summary&gt;
public static void CaptureFrom(UnboxSim sim, G17Achievements achievements, int coins, long nowUnix, G17Profile p
{
    if (profile == null) throw new ArgumentNullException(nameof(profile));
    profile.albumIds = new List&lt;string&gt;();
    if (sim != null &amp;&amp; sim.AlbumIds != null)
    {
        foreach (var id in sim.AlbumIds)
        {
            if (!string.IsNullOrEmpty(id)) profile.albumIds.Add(id);
        }
        profile.albumIds.Sort(StringComparer.Ordinal);
    }
    profile.coins = coins &lt; 0 ? 0 : coins;
    profile.achievementsUnlocked = achievements != null
        ? new List&lt;string&gt;(achievements.SnapshotUnlocked())
        : new List&lt;string&gt;();
    profile.revealDayStreak = sim != null ? sim.RevealDayStreak : 0;
    if (profile.revealDayStreak &lt; 0) profile.revealDayStreak = 0;
    profile.longestRevealDayStreak = sim != null ? sim.LongestRevealDayStreak : 0;
    if (profile.longestRevealDayStreak &lt; profile.revealDayStreak) profile.longestRevealDayStreak = profile.re
    profile.lastRealDayUnix = nowUnix &lt; 0 ? 0 : nowUnix;
}
// ---------------- serialize / persist / load ----------------
/// &lt;summary&gt;Serialize the profile into the single-key JSON payload.&lt;/summary&gt;
public static string Serialize(G17Profile profile)
{
    if (profile == null) throw new ArgumentNullException(nameof(profile));
    var data = new Dictionary&lt;string, object&gt;
    {
        { VersionKey, (long)CurrentVersion },
        { "album", new List&lt;object&gt;(profile.albumIds ?? new List&lt;string&gt;()) },
        { "coins", (long)(profile.coins &lt; 0 ? 0 : profile.coins) },
        { "achv", new List&lt;object&gt;(profile.achievementsUnlocked ?? new List&lt;string&gt;()) },
        { "streak", (long)(profile.revealDayStreak &lt; 0 ? 0 : profile.revealDayStreak) },
        { "streakBest", (long)(profile.longestRevealDayStreak &lt; 0 ? 0 : profile.longestRevealDayStreak) },
        { "dayUnix", profile.lastRealDayUnix &lt; 0 ? 0L : profile.lastRealDayUnix },
    };
    return MiniJson.Serialize(data);
}
/// &lt;summary&gt;Write the profile through the save service (local authority).&lt;/summary&gt;
public static void Persist(ISaveService save, G17Profile profile, string slot = Slot)
{
    if (save == null) return;
    save.Save(slot, Serialize(profile));
}
/// &lt;summary&gt;
```

```
/// Load the slot into the given profile. Never throws on bad data:
/// corrupted or future-version payloads reset the profile to fresh
/// defaults. Both NewGame and CorruptedReset reset deterministically
/// first, so a stale caller-side profile never survives either fresh
/// outcome (G12 parity).
/// &lt;/summary&gt;
public static G17LoadStatus LoadInto(ISaveService save, G17Profile profile, string slot = Slot)
{
    if (save == null) throw new ArgumentNullException(nameof(save));
    if (profile == null) throw new ArgumentNullException(nameof(profile));
    string raw;
    try
    {
        raw = save.Load(slot);
    }
    catch (Exception e)
    {
        // The service itself chokes on a corrupted payload - degrade, never crash the boot.
        Logger.Warn("G17.Save", "corrupt payload at service level, resetting slot: " + e.Message);
        return Fresh(profile);
    }
    if (string.IsNullOrEmpty(raw))
    {
        ResetProfile(profile);
        return G17LoadStatus.NewGame;
    }
    Dictionary&lt;string, object&gt; data;
    try
    {
        data = MiniJson.Deserialize(raw) as Dictionary&lt;string, object&gt;;
    }
    catch (Exception e)
    {
        Logger.Warn("G17.Save", "corrupt payload, resetting slot: " + e.Message);
        return Fresh(profile);
    }
    if (data == null)
    {
        Logger.Warn("G17.Save", "non-dict payload, resetting slot");
        return Fresh(profile);
    }
    int version = ReadInt(data, VersionKey, 0);
    if (version &gt; CurrentVersion)
    {
        Logger.Warn("G17.Save", "future version " + version + " &gt; " + CurrentVersion + ", resetting slot");
        return Fresh(profile);
    }
    profile.albumIds = ReadAlbumIds(data);
    profile.coins = ReadInt(data, "coins", 0);
    if (profile.coins &lt; 0) profile.coins = 0;
```

```
    profile.achievementsUnlocked = ReadAchievementIds(data);
    profile.revealDayStreak = ReadInt(data, "streak", 0);
    if (profile.revealDayStreak &lt; 0) profile.revealDayStreak = 0;
    profile.longestRevealDayStreak = ReadInt(data, "streakBest", 0);
    if (profile.longestRevealDayStreak &lt; profile.revealDayStreak) profile.longestRevealDayStreak = profile.re
    profile.lastRealDayUnix = ReadLong(data, "dayUnix", 0);
    if (profile.lastRealDayUnix &lt; 0) profile.lastRealDayUnix = 0;
    return version &lt; CurrentVersion ? G17LoadStatus.Migrated : G17LoadStatus.Loaded;
}
static G17LoadStatus Fresh(G17Profile profile)
{
    ResetProfile(profile);
    return G17LoadStatus.CorruptedReset;
}
static void ResetProfile(G17Profile profile)
{
    profile.albumIds = new List&lt;string&gt;();
    profile.coins = 0;
    profile.achievementsUnlocked = new List&lt;string&gt;();
    profile.revealDayStreak = 0;
    profile.longestRevealDayStreak = 0;
    profile.lastRealDayUnix = 0;
}
/// &lt;summary&gt;Structural + catalog clamp: null/empty rows drop,
/// duplicates collapse, ids outside the frozen album table drop
/// (tamper face - the catalog is the only id authority).&lt;/summary&gt;
static List&lt;string&gt; ReadAlbumIds(Dictionary&lt;string, object&gt; data)
{
    var list = new List&lt;string&gt;();
    if (!data.TryGetValue("album", out var value) || !(value is System.Collections.IEnumerable rows)) return lis
    var seen = new HashSet&lt;string&gt;();
    foreach (var row in rows)
    {
        string id = row as string;
        if (string.IsNullOrEmpty(id) || !seen.Add(id)) continue;
        if (IsKnownAlbumId(id)) list.Add(id);
        else Logger.Warn("G17.Save", "album id outside catalog, dropped: " + id);
    }
    list.Sort(StringComparer.Ordinal);
    return list;
}
static bool IsKnownAlbumId(string id)
{
    var catalog = G17AlbumCatalog.Load();
    if (catalog == null || catalog.Items == null) return false;
    for (int i = 0; i &lt; catalog.Items.Length; i++)
    {
        if (catalog.Items[i] != null &amp;&amp; catalog.Items[i].id == id) return true;
    }
    return false;
```

```
    }
    /// &lt;summary&gt;Frozen-eight clamp: only ids of the def table survive,
    /// duplicates collapse, order = definition order (stable payload).&lt;/summary&gt;
    static List&lt;string&gt; ReadAchievementIds(Dictionary&lt;string, object&gt; data)
    {
        var list = new List&lt;string&gt;();
        if (!data.TryGetValue("achv", out var value) || !(value is System.Collections.IEnumerable rows)) return list
        var present = new HashSet&lt;string&gt;();
        foreach (var row in rows)
        {
            string id = row as string;
            if (!string.IsNullOrEmpty(id)) present.Add(id);
        }
        var defs = G17Achievements.All;
        for (int i = 0; i &lt; defs.Count; i++)
        {
            if (present.Contains(defs[i].Id)) list.Add(defs[i].Id);
        }
        return list;
    }
    static int ReadInt(Dictionary&lt;string, object&gt; data, string key, int fallback)
    {
        if (!data.TryGetValue(key, out var value) || value == null) return fallback;
        try
        {
            return Convert.ToInt32(Convert.ToDouble(value));
        }
        catch
        {
            return fallback;
        }
    }
    static long ReadLong(Dictionary&lt;string, object&gt; data, string key, long fallback)
    {
        if (!data.TryGetValue(key, out var value) || value == null) return fallback;
        try
        {
            return Convert.ToInt64(Convert.ToDouble(value));
        }
        catch
        {
            return fallback;
        }
    }
}
/// &lt;summary&gt;
/// Editor-check wrapper face for the save domain (task S2-T004 slice A,
/// G12SaveSchema paradigm): CoreCheck-style verdict faces mirror the
/// G17SaveTests coverage so the smoke chain owns the save DoD
/// (round-trip + four load states + tamper clamps + calendar policy)
```

```
/// in machine-readable form. Every face is pure C# and EditMode-pinnable.
/// &lt;/summary&gt;
public static class G17SaveSchema
{
    /// &lt;summary&gt;In-memory ISaveService stub (check path only, zero platform coupling).&lt;/summary&gt;
    sealed class MemorySaveService : ISaveService
    {
        readonly Dictionary&lt;string, string&gt; slots = new Dictionary&lt;string, string&gt;();
        public void Save(string slot, string payloadJson) { slots[slot] = payloadJson; }
        public string Load(string slot)
        {
            string v;
            return slots.TryGetValue(slot, out v) ? v : null;
        }
        public SyncResult CloudSync() { return new SyncResult(false, null); }
    }
    static G17Profile MakeSeededProfile()
    {
        return new G17Profile
        {
            albumIds = new List&lt;string&gt; { "C01", "R01" },
            coins = 42,
            achievementsUnlocked = new List&lt;string&gt; { G17Achievements.FirstUnbox, G17Achievements.BoxClub },
            revealDayStreak = 3,
            longestRevealDayStreak = 5,
            lastRealDayUnix = 1700000000,
        };
    }
    /// &lt;summary&gt;Serialize - persist - reload keeps every field and re-serializes identically.&lt;/summary&gt;
    public static bool RoundTripPreservesPayload()
    {
        var save = new MemorySaveService();
        var seeded = MakeSeededProfile();
        string once = G17SaveSystem.Serialize(seeded);
        G17SaveSystem.Persist(save, seeded);
        var back = new G17Profile();
        if (G17SaveSystem.LoadInto(save, back) != G17LoadStatus.Loaded) return false;
        if (G17SaveSystem.Serialize(back) != once) return false;
        return back.coins == 42
            &amp;&amp; back.revealDayStreak == 3
            &amp;&amp; back.longestRevealDayStreak == 5
            &amp;&amp; back.lastRealDayUnix == 1700000000
            &amp;&amp; back.albumIds.Count == 2 &amp;&amp; back.albumIds.Contains("C01") &amp;&amp; back.albumIds.Co
            &amp;&amp; back.achievementsUnlocked.Count == 2
            &amp;&amp; back.achievementsUnlocked[0] == G17Achievements.FirstUnbox
            &amp;&amp; back.achievementsUnlocked[1] == G17Achievements.BoxClub;
    }
    /// &lt;summary&gt;Empty slot = NewGame, and the reset wipes any stale caller-side state.&lt;/summary&gt;
    public static bool StateNewGameOnEmptySlot()
    {
```

```
    var save = new MemorySaveService();
    var stale = MakeSeededProfile();
    if (G17SaveSystem.LoadInto(save, stale) != G17LoadStatus.NewGame) return false;
    return stale.albumIds.Count == 0 &amp;&amp; stale.coins == 0 &amp;&amp; stale.achievementsUnlocked.Count ==
        &amp;&amp; stale.revealDayStreak == 0 &amp;&amp; stale.lastRealDayUnix == 0;
}
/// &lt;summary&gt;Version-less payload (pre-version build) migrates by filling defaults.&lt;/summary&gt;
public static bool StateMigratedOnVersionlessPayload()
{
    var save = new MemorySaveService();
    save.Save(G17SaveSystem.Slot,
        "{\"album\":[\"C01\"],\"coins\":7,\"achv\":[\"first_unbox\"],\"streak\":2,\"streakBest\":4,\"dayUnix\":1
    var profile = new G17Profile();
    if (G17SaveSystem.LoadInto(save, profile) != G17LoadStatus.Migrated) return false;
    return profile.coins == 7 &amp;&amp; profile.albumIds.Count == 1 &amp;&amp; profile.albumIds[0] == "C01"
        &amp;&amp; profile.achievementsUnlocked.Count == 1
        &amp;&amp; profile.revealDayStreak == 2 &amp;&amp; profile.longestRevealDayStreak == 4
        &amp;&amp; profile.lastRealDayUnix == 1700000000;
}
/// &lt;summary&gt;Unparseable payload degrades to a fresh profile instead of crashing the boot.&lt;/summary&gt;
public static bool StateCorruptedResetOnGarbage()
{
    var save = new MemorySaveService();
    save.Save(G17SaveSystem.Slot, "{ not json at all");
    var stale = MakeSeededProfile();
    if (G17SaveSystem.LoadInto(save, stale) != G17LoadStatus.CorruptedReset) return false;
    return stale.albumIds.Count == 0 &amp;&amp; stale.coins == 0 &amp;&amp; stale.achievementsUnlocked.Count ==
}
/// &lt;summary&gt;A payload from a newer build resets rather than guessing at its schema.&lt;/summary&gt;
public static bool StateCorruptedResetOnFutureVersion()
{
    var save = new MemorySaveService();
    save.Save(G17SaveSystem.Slot, "{\"g17v\":99,\"album\":[],\"coins\":0,\"achv\":[],\"streak\":0,\"streakBest\"
    var profile = new G17Profile();
    return G17SaveSystem.LoadInto(save, profile) == G17LoadStatus.CorruptedReset;
}
/// &lt;summary&gt;Tamper clamps: unknown album/achievement ids drop, negative
/// coins/streaks/timestamps clamp at zero, load still succeeds.&lt;/summary&gt;
public static bool TamperClampsDropUnknownAndNegativeRows()
{
    var save = new MemorySaveService();
    save.Save(G17SaveSystem.Slot,
        "{\"g17v\":1,\"album\":[\"C01\",\"ZZ99_tampered\",\"\"],\"coins\":-5,\"achv\":[\"first_unbox\",\"hacked_
    var profile = new G17Profile();
    if (G17SaveSystem.LoadInto(save, profile) != G17LoadStatus.Loaded) return false;
    return profile.albumIds.Count == 1 &amp;&amp; profile.albumIds[0] == "C01"
        &amp;&amp; profile.coins == 0
        &amp;&amp; profile.achievementsUnlocked.Count == 1 &amp;&amp; profile.achievementsUnlocked[0] == G17Achi
        &amp;&amp; profile.revealDayStreak == 0 &amp;&amp; profile.longestRevealDayStreak == 0
        &amp;&amp; profile.lastRealDayUnix == 0;
```

```
        }
        /// &lt;summary&gt;Pure calendar policy faces: day index clamps, gap policy honest.&lt;/summary&gt;
        public static bool CalendarPolicyFacesHonest()
        {
            if (G17SaveSystem.RealDayIndexOf(0) != 0) return false;
            if (G17SaveSystem.RealDayIndexOf(-5) != 0) return false;
            if (G17SaveSystem.RealDayIndexOf(G17SaveSystem.RealDaySeconds * 3) != 3) return false;
            if (G17SaveSystem.StreakAfterGap(4, 0) != 4) return false;
            if (G17SaveSystem.StreakAfterGap(4, 1) != 4) return false;
            if (G17SaveSystem.StreakAfterGap(4, 2) != 0) return false;
            if (G17SaveSystem.StreakAfterGap(7, 30) != 0) return false;
            if (G17SaveSystem.StreakAfterGap(-2, 1) != 0) return false;
            return true;
        }
        /// &lt;summary&gt;Boot seed mapping: same-day keeps with today on record,
        /// next-day keeps with yesterday on record, skipped day and a
        /// never-saved profile both start fresh at -1.&lt;/summary&gt;
        public static bool BootSeedFacesHonest()
        {
            long saved = 1700000000;
            int savedDay = (int)(saved / G17SaveSystem.RealDaySeconds);
            long dayStart = (long)savedDay * G17SaveSystem.RealDaySeconds;
            var profile = MakeSeededProfile();   // revealDayStreak = 3
            int cur;
            int last;
            G17SaveSystem.StreakSeedForBoot(profile, dayStart + 5, out cur, out last);
            if (cur != 3 || last != savedDay) return false;
            G17SaveSystem.StreakSeedForBoot(profile, dayStart + G17SaveSystem.RealDaySeconds, out cur, out last);
            if (cur != 3 || last != savedDay) return false;
            G17SaveSystem.StreakSeedForBoot(profile, dayStart + 3 * G17SaveSystem.RealDaySeconds, out cur, out last);
            if (cur != 0 || last != -1) return false;
            G17SaveSystem.StreakSeedForBoot(new G17Profile(), dayStart, out cur, out last);
            return cur == 0 &amp;&amp; last == -1;
        }
    }
}
// G17Sfx.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.Unbox
{
    /// &lt;summary&gt;
    /// Procedural sfx + signature hook for G17 (task S1-T006, X028 earworm
    /// red line: hook hummable within 3s, unique across the portfolio, zero
    /// shipped audio files - pure math over float[] buffers, deterministic,
    /// EditMode testable, no Random, no files; G12Sfx three-axis paradigm).
    ///
    /// G17 signature hook "pop-pop-pop + ding-dong": three bubble-wrap pops in
```

```
/// an accelerating train, answered by a marimba double tone that FALLS
/// (C5 struck, then G4 below it). It mirrors the unboxing fantasy - pop the
/// film, then the reveal lands lower and warmer, like the item settling into
/// your hands - and stays mutually exclusive against the sibling hooks:
/// G02 chime rings upward, G03 jelly wobbles down in three steps, G06
/// glides then knocks, G12 tightens then bursts, and G16's money line is a
/// RISING three-note figure; G17 is the portfolio's only struck falling
/// PAIR over a noisy pop train (two clean struck notes, never three, never
/// rising). The same recipes feed the per-beat feedback (seal_cut /
/// bubble_pop / reveal_rare), so the signature is heard organically inside
/// the opening: the first three pops of the first package play the pop
/// train, and the first forced-Rare reveal lands the ding-dong - the hook
/// promises exactly what the game plays.
/// &lt;/summary&gt;
public static class G17Sfx
{
    public const int SampleRate = 22050;
    public const string SealCut = "seal_cut";       // strip tear rip (sss-la)
    public const string BubblePop = "bubble_pop";  // membrane burst (pa)
    public const string RevealRare = "reveal_rare"; // layered fanfare (ding-dong + shimmer + pad)
    public const string MarimbaFall = "marimba_fall"; // the struck falling pair (ding-dong)
    public const string LatchPry = "latch_pry";    // S3-T005: ratchet clack (catch resists, tab gives)
    public const string LinerOpen = "liner_open";  // S3-T005: paper liner rustle sweep (soft tissue pull)
    public const string DoubleOpen = "double_open"; // S3-T005: twin same-pitch knock (the doubles offer)
    public const string Leitmotif = "leitmotif";   // signature hook: pop-pop-pop + ding-dong
    public const double LeitmotifGap1Sec = 0.030;  // pop .. pop (the train accelerates)
    public const double LeitmotifGap2Sec = 0.022;  // pop-pop (tightest link)
    public const double LeitmotifGap3Sec = 0.055;  // held breath before the reveal lands
    // ---- delivery caliber (G12Sfx S3-T002 family law, zero-file family) --
    // Same-batch loudness spread &lt;= 1 dB measured K-weighted (BS.1770),
    // sample peak &lt;= -3.6 dBFS so true peak keeps the -3 dB spec with
    // inter-sample overshoot margin; the -16 LUFS anchor belongs to the
    // 60 s BGM row (AU work order, MCP face), not to these short acts.
    public const double SamplePeakCap = 0.66;     // -3.6 dBFS sample peak
    public const double FamilySpreadMaxDb = 1.0;  // same-batch loudness difference &lt;= 1 dB
    static readonly Dictionary&lt;string, AudioClip&gt; ClipCache = new();
    static readonly Dictionary&lt;string, float[]&gt; _calibratedActs = new();
    static bool _familySettled;
    static AudioSource _host;   // bus sfx channel: fire-and-forget one-shots
    static AudioSource _bgm;    // bus bgm channel: loop slot for the AU bgm row
    /// &lt;summary&gt;Deterministic sample synthesis for one clip id (pure, no
    /// Unity objects). The four acts are loudness-calibrated as one family
    /// (EnsureFamily); every render is an independent identical copy, so
    /// callers may keep and compare buffers freely.&lt;/summary&gt;
    public static float[] Synth(string clipId)
    {
        switch (clipId)
        {
            case SealCut:
            case BubblePop:
```

```
            case RevealRare:
            case MarimbaFall:
            case LatchPry:
            case LinerOpen:
            case DoubleOpen:
                EnsureFamily();
                return Copy(_calibratedActs[clipId]);
            case Leitmotif: return LeitmotifSamples();
            default: return new float[0];
        }
    }
    /// &lt;summary&gt;Cached AudioClip wrapper for playback (main thread only).&lt;/summary&gt;
    public static AudioClip Clip(string clipId)
    {
        if (ClipCache.TryGetValue(clipId, out var cached) &amp;&amp; cached != null) return cached;
        var data = Synth(clipId);
        if (data.Length == 0) return null;
        var clip = AudioClip.Create("g17_sfx_" + clipId, data.Length, 1, SampleRate, false);
        clip.SetData(data, 0);
        ClipCache[clipId] = clip;
        return clip;
    }
    /// &lt;summary&gt;Fire-and-forget playback on the sfx channel of the audio
    /// bus; never throws in headless runs.&lt;/summary&gt;
    public static void Play(string clipId)
    {
        PlayPitched(clipId, 0f);
    }
    /// &lt;summary&gt;Fire-and-forget playback with a pitch factor (the bubble
    /// pitch-ladder feel anchor 1.00 + 0.02 * popIndex rides here): a
    /// transient voice source carries the pitch so concurrent pops each
    /// keep their own step, and the voice destroys itself after the clip
    /// ends. Non-positive pitch = unity. Never throws in headless runs.&lt;/summary&gt;
    public static void PlayPitched(string clipId, float pitch)
    {
        try
        {
            EnsureBus();
            var clip = Clip(clipId);
            if (clip == null) return;
            var go = new GameObject("G17SfxVoice");
            UnityEngine.Object.DontDestroyOnLoad(go);
            var src = go.AddComponent&lt;AudioSource&gt;();
            src.playOnAwake = false;
            if (pitch &gt; 0f) src.pitch = pitch;
            src.PlayOneShot(clip);
            UnityEngine.Object.Destroy(go, clip.length + 0.10f);
        }
        catch (Exception e)
        {
```

```
            Logger.Warn("G17.Sfx", "play skipped (" + clipId + "): " + e.Message);
        }
    }
    /// &lt;summary&gt;BGM channel of the bus, reserved for the AU bgm row (AU02
    /// work order, MCP face): looping, null-safe, honest false while the
    /// asset is not in - wiring once, zero rework on arrival (G12Sfx law).&lt;/summary&gt;
    public static bool AttachBgm(AudioClip clip)
    {
        try
        {
            if (clip == null) return false;
            EnsureBus();
            _bgm.clip = clip;
            _bgm.Play();
            return true;
        }
        catch (Exception e)
        {
            Logger.Warn("G17.Sfx", "bgm attach skipped: " + e.Message);
            return false;
        }
    }
    /// &lt;summary&gt;One hidden bus host carries both channels (sfx one-shots +
    /// looping bgm slot), so the whole audio face owns exactly one scene
    /// object; headless-safe because the first call happens from gameplay
    /// code, never from synthesis or tests.&lt;/summary&gt;
    static void EnsureBus()
    {
        if (_host != null) return;
        var go = new GameObject("G17AudioBus");
        UnityEngine.Object.DontDestroyOnLoad(go);
        _host = go.AddComponent&lt;AudioSource&gt;();
        _bgm = go.AddComponent&lt;AudioSource&gt;();
        _bgm.loop = true;
        _bgm.playOnAwake = false;
    }
    // ---- loudness meter + family calibration (G12Sfx S3-T002 law) --------
    /// &lt;summary&gt;K-weighted loudness in LUFS for one buffer. The file face
    /// (ffmpeg loudnorm / libebur128) gates 400 ms blocks; the procedural
    /// acts are shorter than one block, so the meter measures the full clip
    /// as a single window - one uniform, comparable basis for the whole
    /// zero-file family. Pure math, no Random, bit-stable across renders.&lt;/summary&gt;
    public static double MeasureLoudnessLufs(float[] buf)
    {
        if (buf == null || buf.Length == 0) return SilenceLufs;
        var y = KWeight(buf);
        double z = 0.0;
        for (int i = 0; i &lt; y.Length; i++) z += (double)y[i] * y[i];
        z /= y.Length;
        if (z &lt; 1e-12) return SilenceLufs;
```

```
    Assert.AreEqual(0, profile.revealDayStreak, "negative streak clamps at zero");
    Assert.AreEqual(0, profile.lastRealDayUnix, "negative offline timestamp clamps at zero");
}
[Test]
public void SV07_SameDayReturnKeepsStreakWithoutStacking()
{
    long saved = 1700000000;
    int savedDay = (int)(saved / G17SaveSystem.RealDaySeconds);
    long dayStart = (long)savedDay * G17SaveSystem.RealDaySeconds;
    var profile = new G17Profile { revealDayStreak = 4, longestRevealDayStreak = 5, lastRealDayUnix = saved };
    int cur;
    int last;
    G17SaveSystem.StreakSeedForBoot(profile, dayStart + 5, out cur, out last);
    Assert.AreEqual(4, cur, "same-day return keeps the streak");
    Assert.AreEqual(savedDay, last, "same-day return puts today on record");
    var sim = new UnboxSim(11);
    sim.SeedRevealDayStreak(cur, 5, last);
    sim.SetProductionDayOverride(delegate { return savedDay; });
    sim.Open();
    WalkOnePackage(sim);
    Assert.AreEqual(4, sim.RevealDayStreak, "a second reveal on the same production day must never stack");
    Assert.AreEqual(5, sim.LongestRevealDayStreak);
}
[Test]
public void SV08_NextDayReturnExtendsStreakOnFirstReveal()
{
    long saved = 1700000000;
    int savedDay = (int)(saved / G17SaveSystem.RealDaySeconds);
    long nextDay = (long)(savedDay + 1) * G17SaveSystem.RealDaySeconds;
    var profile = new G17Profile { revealDayStreak = 4, longestRevealDayStreak = 5, lastRealDayUnix = saved };
    int cur;
    int last;
    G17SaveSystem.StreakSeedForBoot(profile, nextDay + 60, out cur, out last);
    Assert.AreEqual(4, cur, "next-day return keeps the streak");
    Assert.AreEqual(savedDay, last, "next-day return keeps yesterday on record so the first reveal extends");
    var sim = new UnboxSim(12);
    sim.SeedRevealDayStreak(cur, 5, last);
    int today = savedDay + 1;
    sim.SetProductionDayOverride(delegate { return today; });
    sim.Open();
    WalkOnePackage(sim);
    Assert.AreEqual(5, sim.RevealDayStreak, "the first reveal of the next production day extends the chain");
    Assert.AreEqual(5, sim.LongestRevealDayStreak);
}
[Test]
public void SV09_SkippedDayResetsStreakToOneOnReveal()
{
    long saved = 1700000000;
    int savedDay = (int)(saved / G17SaveSystem.RealDaySeconds);
    long skipped = (long)(savedDay + 3) * G17SaveSystem.RealDaySeconds;
```

```
    var profile = new G17Profile { revealDayStreak = 4, longestRevealDayStreak = 5, lastRealDayUnix = saved };
    int cur;
    int last;
    G17SaveSystem.StreakSeedForBoot(profile, skipped, out cur, out last);
    Assert.AreEqual(0, cur, "a skipped day resets the streak");
    Assert.AreEqual(-1, last, "a skipped day leaves no day on record");
    var sim = new UnboxSim(13);
    sim.SeedRevealDayStreak(cur, 5, last);
    int today = savedDay + 3;
    sim.SetProductionDayOverride(delegate { return today; });
    sim.Open();
    WalkOnePackage(sim);
    Assert.AreEqual(1, sim.RevealDayStreak, "the first reveal after a skipped day starts a fresh chain at one");
    Assert.AreEqual(5, sim.LongestRevealDayStreak, "the longest streak is a ratchet and never drops");
}
[Test]
public void SV10_CaptureFromMirrorsLiveTruthsAndRoundTrips()
{
    var sim = new UnboxSim(21);
    var achievements = new G17Achievements(sim);
    sim.Open();
    WalkOnePackage(sim);
    achievements.Evaluate();
    long now = 1758600000;
    var profile = new G17Profile();
    G17SaveSystem.CaptureFrom(sim, achievements, 42, now, profile);
    var expected = new List&lt;string&gt;(sim.AlbumIds);
    expected.Sort(StringComparer.Ordinal);
    Assert.AreEqual(expected, profile.albumIds, "capture mirrors the sim album set");
    Assert.AreEqual(42, profile.coins);
    Assert.IsTrue(profile.achievementsUnlocked.Contains(G17Achievements.FirstUnbox), "capture mirrors the unlock
    Assert.AreEqual(sim.RevealDayStreak, profile.revealDayStreak);
    Assert.AreEqual(sim.LongestRevealDayStreak, profile.longestRevealDayStreak);
    Assert.AreEqual(now, profile.lastRealDayUnix);
    // full chain: capture -&gt; persist -&gt; load -&gt; identical payload
    var save = new MemorySaveService();
    string once = G17SaveSystem.Serialize(profile);
    G17SaveSystem.Persist(save, profile);
    var back = new G17Profile();
    Assert.AreEqual(G17LoadStatus.Loaded, G17SaveSystem.LoadInto(save, back));
    Assert.AreEqual(once, G17SaveSystem.Serialize(back));
}
[Test]
public void SV11_AchievementsRestoreNoRetostSnapshotOrdered()
{
    var achievements = new G17Achievements(new UnboxSim(31));
    achievements.RestoreUnlocked(new[] { G17Achievements.FirstUnbox, "hacked_id", null, "" });
    Assert.AreEqual(1, achievements.UnlockedCount, "only frozen-eight ids survive the restore");
    Assert.IsTrue(achievements.IsUnlocked(G17Achievements.FirstUnbox));
    var snap = achievements.SnapshotUnlocked();
```

```
            Assert.AreEqual(1, snap.Length);
            Assert.AreEqual(G17Achievements.FirstUnbox, snap[0]);
            // no re-toast: a later Evaluate on real truths stays silent for restored ids
            var sim = new UnboxSim(32);
            var restored = new G17Achievements(sim);
            restored.RestoreUnlocked(new[] { G17Achievements.FirstUnbox });
            sim.Open();
            WalkOnePackage(sim);
            var toasts = restored.Evaluate();
            for (int i = 0; i &lt; toasts.Length; i++)
            {
                Assert.AreNotEqual(G17Achievements.FirstUnbox, toasts[i].Id, "a restored id must never re-toast");
            }
            // unlock order: fresh truths append after restored ids in definition order
            var ordered = restored.SnapshotUnlocked();
            Assert.AreEqual(G17Achievements.FirstUnbox, ordered[0]);
            for (int i = 1; i &lt; ordered.Length; i++)
            {
                Assert.AreNotEqual(ordered[i - 1], ordered[i], "snapshot ids are unique and definition-ordered");
            }
        }
    }
}
// G17SfxTests.cs
using System;
using NUnit.Framework;
using Sy.Unbox;
namespace Sy.Unbox.Tests
{
    /// &lt;summary&gt;
    /// Determinism + X028 discipline tests for the procedural sfx (task S1-T006,
    /// G12SfxTests paradigm): synthesis is pure math (no Random, no files) so
    /// every render is identical; the signature hook fits the 3s hummable
    /// window and is the exact composition of its pop train and marimba fall;
    /// the loudness family keeps the loudnorm caliber; the three axes stay
    /// structurally distinct voices (burst / rip / struck pair); and the
    /// marimba tail is a struck FALLING PAIR - two clean notes stepping down,
    /// never three (G03's jelly), never rising (G16's money line).
    /// &lt;/summary&gt;
    public class G17SfxTests
    {
        static int ZeroCrossings(float[] buf, int from, int to)
        {
            int n = 0;
            int end = Math.Min(to, buf.Length);
            for (int i = Math.Max(1, from); i &lt; end; i++)
                if ((buf[i - 1] &lt;= 0f &amp;&amp; buf[i] &gt; 0f) || (buf[i - 1] &gt;= 0f &amp;&amp; buf[i] &lt; 0f))
            return n;
        }
        /// &lt;summary&gt;Time (seconds) at which cumulative energy crosses 50%.&lt;/summary&gt;
```

```
static double HalfEnergySec(float[] buf)
{
    double total = 0.0;
    for (int i = 0; i &lt; buf.Length; i++) total += (double)buf[i] * buf[i];
    if (total &lt;= 0.0) return 0.0;
    double acc = 0.0;
    for (int i = 0; i &lt; buf.Length; i++)
    {
        acc += (double)buf[i] * buf[i];
        if (acc &gt;= total * 0.5) return i / (double)G17Sfx.SampleRate;
    }
    return buf.Length / (double)G17Sfx.SampleRate;
}
/// &lt;summary&gt;Distinct amplitude groups above 0.60 x peak, re-armed below
/// 0.40 x peak (G12 creak pulse-train counting law, 1 ms max window).&lt;/summary&gt;
static int CountGroups(float[] buf)
{
    double peak = 0.0;
    int win = Math.Max(1, G17Sfx.SampleRate / 1000);
    for (int i = 0; i &lt; buf.Length; i++) peak = Math.Max(peak, Math.Abs(buf[i]));
    int groups = 0;
    bool armed = true;
    for (int i = 0; i &lt; buf.Length; i += win)
    {
        double e = 0.0;
        int end = Math.Min(buf.Length, i + win);
        for (int k = i; k &lt; end; k++) e = Math.Max(e, Math.Abs(buf[k]));
        if (armed &amp;&amp; e &gt; 0.60 * peak) { groups++; armed = false; }
        else if (!armed &amp;&amp; e &lt; 0.40 * peak) armed = true;
    }
    return groups;
}
[Test]
public void Synth_Is_Deterministic_And_Bounded_For_Every_Clip()
{
    string[] ids = { G17Sfx.SealCut, G17Sfx.BubblePop, G17Sfx.RevealRare, G17Sfx.MarimbaFall, G17Sfx.LatchPry, G
    foreach (var id in ids)
    {
        var a = G17Sfx.Synth(id);
        var b = G17Sfx.Synth(id);
        Assert.Greater(a.Length, 0, "clip must synthesize: " + id);
        Assert.AreEqual(a.Length, b.Length, "the same recipe keeps one length: " + id);
        for (int i = 0; i &lt; a.Length; i++)
        {
            Assert.AreEqual(a[i], b[i], "synthesis must be deterministic (no Random): " + id);
            Assert.IsFalse(float.IsNaN(a[i]) || float.IsInfinity(a[i]), "samples must be finite: " + id);
            Assert.LessOrEqual(Math.Abs(a[i]), 1f, "normalized samples stay inside [-1,1]: " + id);
            Assert.LessOrEqual(Math.Abs(a[i]), (float)G17Sfx.SamplePeakCap + 1e-4f,
                "TP -3 dB caliber: the loudness family caps sample peak: " + id);
        }
```

```
    }
    // unknown ids fold to silence, never throw
    Assert.AreEqual(0, G17Sfx.Synth("no_such_clip").Length, "unknown clip ids synthesize empty");
}
[Test]
public void Signature_Hook_Is_The_Exact_Composition_Of_Pop_Train_And_Fall()
{
    var hook = G17Sfx.Synth(G17Sfx.Leitmotif);
    double sec = hook.Length / (double)G17Sfx.SampleRate;
    Assert.LessOrEqual(sec, 3.0, "X028: the hook must be hummable within 3s");
    Assert.GreaterOrEqual(sec, 0.30, "the hook must be a real phrase, not a blip");
    // the signature is literally pop + gap + pop + gap + pop + gap + fall:
    // the exact composition is the assertion oracle (deterministic buffers)
    var pop = G17Sfx.Synth(G17Sfx.BubblePop);
    var fall = G17Sfx.Synth(G17Sfx.MarimbaFall);
    int gap1 = (int)(G17Sfx.SampleRate * G17Sfx.LeitmotifGap1Sec);
    int gap2 = (int)(G17Sfx.SampleRate * G17Sfx.LeitmotifGap2Sec);
    int gap3 = (int)(G17Sfx.SampleRate * G17Sfx.LeitmotifGap3Sec);
    Assert.AreEqual(pop.Length + gap1 + pop.Length + gap2 + pop.Length + gap3 + fall.Length, hook.Length,
        "the leitmotif must contain the accelerating pop train then the falling pair");
    // segment-wise equality: the phrase is the exact calibrated acts the
    // game plays, sample for sample - no phrase-level rescale
    int p2 = pop.Length + gap1;
    int p3 = p2 + pop.Length + gap2;
    int fallAt = p3 + pop.Length + gap3;
    for (int i = 0; i &lt; pop.Length; i++)
    {
        Assert.AreEqual(pop[i], hook[i], "hook pop1 segment must equal the calibrated pop act at " + i);
        Assert.AreEqual(pop[i], hook[p2 + i], "hook pop2 segment must equal the calibrated pop act at " + i);
        Assert.AreEqual(pop[i], hook[p3 + i], "hook pop3 segment must equal the calibrated pop act at " + i);
    }
    for (int i = 0; i &lt; fall.Length; i++)
        Assert.AreEqual(fall[i], hook[fallAt + i], "hook fall segment must equal the calibrated fall act at " +
    // the train accelerates: gap1 &gt; gap2 (the pops tighten before the fall)
    Assert.Greater(G17Sfx.LeitmotifGap1Sec, G17Sfx.LeitmotifGap2Sec,
        "the pop train must accelerate (pop .. pop-pop), never crawl");
}
[Test]
public void Loudness_Family_Matches_Loudnorm_Caliber_Spread_And_Meter_Scale()
{
    string[] acts = { G17Sfx.SealCut, G17Sfx.BubblePop, G17Sfx.RevealRare, G17Sfx.MarimbaFall, G17Sfx.LatchPry,
    double min = double.MaxValue, max = double.MinValue;
    foreach (var id in acts)
    {
        var b = G17Sfx.Synth(id);
        double lufs = G17Sfx.MeasureLoudnessLufs(b);
        min = Math.Min(min, lufs);
        max = Math.Max(max, lufs);
    }
    Assert.LessOrEqual(max - min, G17Sfx.FamilySpreadMaxDb,
```

```
            "same-batch loudness difference must stay within 1 dB (spread="
            + (max - min).ToString("F3") + " dB)");
        // the meter itself keeps the BS.1770 scale: a -20 dBFS 1 kHz sine is
        // the EBU calibration face and must read about -23 LUFS (K-weighting
        // at 1 kHz + the -0.691 offset cancel to the R128 reference)
        var tone = new float[G17Sfx.SampleRate];
        for (int i = 0; i &lt; tone.Length; i++)
            tone[i] = (float)(Math.Sin(2.0 * Math.PI * 1000.0 * i / G17Sfx.SampleRate) * Math.Pow(10.0, -20.0 / 20.0
        Assert.AreEqual(-23.0, G17Sfx.MeasureLoudnessLufs(tone), 0.35,
            "K-weighted meter keeps the BS.1770 scale (1 kHz sine at -20 dBFS reads about -23 LUFS)");
    }
    [Test]
    public void Three_Axes_Stay_Structurally_Distinct_Voices()
    {
        // burst vs rip vs struck pair: the pop is ONE amplitude group that
        // spends its energy almost immediately; the seal tear is a long
        // crackle train of many groups; the marimba fall is exactly two
        // struck notes that ring. Three different structures, three
        // different voices - the hook's ingredients never blur.
        var pop = G17Sfx.Synth(G17Sfx.BubblePop);
        var cut = G17Sfx.Synth(G17Sfx.SealCut);
        var fall = G17Sfx.Synth(G17Sfx.MarimbaFall);
        Assert.AreEqual(1, CountGroups(pop), "the pop must read as a single burst, never a train");
        Assert.GreaterOrEqual(CountGroups(cut), 5, "the tear must read as a crackle train, not one moan");
        Assert.AreEqual(2, CountGroups(fall), "the fall must read as exactly two struck notes (ding-dong)");
        // burst vs ring: the pop reaches half its energy far faster than the
        // struck marimba pair - a membrane burst against a ringing bar
        double popHalf = HalfEnergySec(pop);
        double fallHalf = HalfEnergySec(fall);
        Assert.Less(popHalf, fallHalf * 0.5,
            "the pop must settle far faster than the marimba ring (burst vs struck tone)");
    }
    [Test]
    public void Marimba_Fall_Is_A_Struck_Falling_Pair_Never_Rising()
    {
        // Portfolio mutual exclusion (X028): G16's money line is a RISING
        // three-note figure and G03's jelly wobbles down in THREE steps -
        // G17's tail must be a struck FALLING PAIR: the second note's
        // dominant contour sits LOWER than the first (fewer crossings per
        // second in the second note's window), so the hook's tonal face
        // steps down, never up, and exactly two notes, never three.
        var fall = G17Sfx.Synth(G17Sfx.MarimbaFall);
        int n = fall.Length;
        int note2At = (int)(G17Sfx.SampleRate * 0.155);   // second strike time (recipe pin)
        double rate1 = ZeroCrossings(fall, 0, note2At) / (note2At / (double)G17Sfx.SampleRate);
        double rate2 = ZeroCrossings(fall, note2At, n) / ((n - note2At) / (double)G17Sfx.SampleRate);
        Assert.Greater(rate1, rate2,
            "the ding-dong must FALL: the second struck note must sit lower than the first (anti-G16 money line)");
        Assert.AreEqual(2, CountGroups(fall), "exactly two struck notes (anti-G03 jelly three-step)");
    }
```

```
        [Test]
        public void Seal_Cut_Rips_Through_Accelerating_Crackle()
        {
            var cut = G17Sfx.Synth(G17Sfx.SealCut);
            int n = cut.Length;
            // the rip brightens as it gives: late crackle crossings &gt; early
            int q1 = ZeroCrossings(cut, 0, n / 4);
            int q4 = ZeroCrossings(cut, 3 * n / 4, n);
            Assert.Greater(q4, q1 * 1.3,
                "the tear must accelerate and brighten as the strip gives way");
            // and it is a train, not one moan
            Assert.GreaterOrEqual(CountGroups(cut), 5,
                "the tear must read as multiple crackle groups");
        }
        [Test]
        public void Bubble_Pop_Reads_As_A_Burst_Not_A_Knock()
        {
            var pop = G17Sfx.Synth(G17Sfx.BubblePop);
            int n = pop.Length;
            double sec = n / (double)G17Sfx.SampleRate;
            // the burst is the attack: near-peak inside 3 ms
            double peak = 0.0;
            for (int i = 0; i &lt; n; i++) peak = Math.Max(peak, Math.Abs(pop[i]));
            double attack = 0.0;
            int aEnd = Math.Min(n, (int)(G17Sfx.SampleRate * 0.003));
            for (int i = 0; i &lt; aEnd; i++) attack = Math.Max(attack, Math.Abs(pop[i]));
            Assert.GreaterOrEqual(attack, 0.75 * peak,
                "the pop must hit hard within its first 3 ms (a burst, not a swell)");
            // the contour settles dark: opens bright, closes low - never a low
            // solid knock
            double openRate = ZeroCrossings(pop, 0, n / 5) / (sec * 0.2);
            double closeRate = ZeroCrossings(pop, n / 2, n) / (sec * 0.5);
            Assert.Greater(openRate, closeRate * 2.0,
                "the pop must open bright and settle dark (burst face, never a knock)");
        }
    }
}
// G17TelemetryTests.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using NUnit.Framework;
using Sy.Platform;
using Sy.Telemetry;
using Sy.Unbox;
using UnityEngine;
namespace Sy.Unbox.Tests
{
    /// &lt;summary&gt;
    /// TelemetryCheck gate (task S2-T007, the 07-doc S2 event family): the
```

```
/// north-star measurement surface in one gate. T01/T02 pin the five-step
/// newbie funnel (canonical order + once-per-session law + the elapsed
/// timeline measured on the real presenter chain), T03 walks the ad dual
/// positions' five-sub-id families end to end on the real
/// TelemetryService + real AdCapPolicy + mock ads (R04/R05 paradigm),
/// T04 enforces the cfg_g17_events zero-missing contract, T05 enforces
/// the registry bidirectional law (the schema names exactly the
/// code-declared vocabulary; family roots stay schema anchors).
/// &lt;/summary&gt;
[TestFixture]
public class G17TelemetryTests
{
    const string DayOne = "20260923";
    /// &lt;summary&gt;Mock ads consuming the same policy instance the funnel
    /// pre-checks from (the suite law: category-aware consume on show).&lt;/summary&gt;
    sealed class MockAds : IAdsService
    {
        readonly AdCapPolicy _policy;
        readonly Func&lt;string&gt; _dayKey;
        public readonly List&lt;string&gt; Shown = new List&lt;string&gt;();
        public MockAds(AdCapPolicy policy, Func&lt;string&gt; dayKey)
        {
            _policy = policy;
            _dayKey = dayKey;
        }
        public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        {
            Shown.Add(pointId);
            string key = _dayKey != null ? _dayKey() : AdCapPolicy.TodayKey();
            bool granted = _policy == null || _policy.TryConsumeRewarded(key, AdCategories.CategoryOfPoint(pointId))
            return Task.FromResult(new RewardResult { Completed = granted });
        }
        public void ShowInterstitial(string pointId) { }
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId) { }
        public event Action&lt;AdLifecycle&gt; OnAdEvent { add { } remove { } }
    }
    static G17UnboxPresenter NewPresenter(int seed)
    {
        return new G17UnboxPresenter(new UnboxSim(seed), G17BoxCatalog.Load(), G17AlbumCatalog.Load());
    }
    /// &lt;summary&gt;Plays whole boxes through the presenter, feeding every
    /// drained command batch to the funnel (the exact production
    /// ApplyCommands wiring face).&lt;/summary&gt;
    static void PlayBoxes(G17UnboxPresenter presenter, G17NewbieFunnel funnel, int boxes)
    {
        int guard = 0;
        int revealed = 0;
        funnel.Observe(presenter.Open());   // caption + guide land on the first batch
        while (revealed &lt; boxes &amp;&amp; guard++ &lt; 600)
```

```
        {
            switch (presenter.Stage)
            {
                case UnboxSim.Stage.Seal:
                    funnel.Observe(presenter.CutSealFromCanvas(Vector2.zero));
                    break;
                case UnboxSim.Stage.Bubble:
                    funnel.Observe(presenter.PopBubbleAt(Vector2.zero));
                    break;
                case UnboxSim.Stage.Latch:
                    funnel.Observe(presenter.PryLatchAt(Vector2.zero));
                    break;
                case UnboxSim.Stage.Liner:
                    funnel.Observe(presenter.OpenLinerTap());
                    break;
                case UnboxSim.Stage.RevealReady:
                    funnel.Observe(presenter.RevealTap(false));
                    revealed++;
                    break;
                default:
                    return;
            }
        }
    }
    [Test]
    public void T01_NewbieFiveSteps_FireOnceInCanonicalOrder_WithElapsedTimeline()
    {
        var sink = new CaptureSink();
        var telemetry = new TelemetryService();
        telemetry.AddSink(sink);
        var funnel = new G17NewbieFunnel(telemetry);
        PlayBoxes(NewPresenter(20260923), funnel, 1);
        Assert.AreEqual(5, funnel.FiredCount,
            "the designed opening fires all five north-star steps");
        CollectionAssert.AreEqual((System.Collections.IEnumerable)G17NewbieFunnel.Steps,
            (System.Collections.IEnumerable)funnel.FiredSteps,
            "the five steps fire in the designed order caption-&gt;first_op-&gt;first_juice-&gt;craft_3-&gt;first_re
        // elapsed timeline: the caption pins t=0 and no step goes back in time
        Assert.AreEqual(0f, funnel.ElapsedOf(G17NewbieFunnel.StepCaption),
            "the caption beat lands at t=0 (H1 pins 0.00s)");
        float prev = -1f;
        foreach (var step in G17NewbieFunnel.Steps)
        {
            float t = funnel.ElapsedOf(step);
            Assert.GreaterOrEqual(t, 0f, step + " must fire on the walk");
            Assert.GreaterOrEqual(t, prev, step + " must not precede the previous step");
            prev = t;
        }
        // the sink saw exactly five evt_newbie_step writes, one per step in order
        Assert.AreEqual(5, sink.Ids.Count, "exactly five newbie writes land on the pipeline");
```

```
    for (int i = 0; i &lt; sink.Ids.Count; i++)
    {
        Assert.AreEqual(EventIds.NewbieStep, sink.Ids[i], "every write rides the unified id");
        Assert.AreEqual(G17NewbieFunnel.Steps[i], sink.Fields[i]["step"],
            "write " + i + " carries its step name");
    }
}
[Test]
public void T02_OncePerSessionLaw_MoreBoxesNeverRefire_FreshSessionRefires()
{
    var sink = new CaptureSink();
    var telemetry = new TelemetryService();
    telemetry.AddSink(sink);
    var funnel = new G17NewbieFunnel(telemetry);
    PlayBoxes(NewPresenter(7), funnel, 3);
    Assert.AreEqual(5, funnel.FiredCount,
        "three boxes later the session still fires each step exactly once");
    Assert.AreEqual(5, sink.Ids.Count, "continuing play books zero re-fire writes");
    // a fresh session (fresh sim + funnel) fires the five steps again:
    // the latch is per session, not global
    var sink2 = new CaptureSink();
    var telemetry2 = new TelemetryService();
    telemetry2.AddSink(sink2);
    var funnel2 = new G17NewbieFunnel(telemetry2);
    PlayBoxes(NewPresenter(8), funnel2, 1);
    Assert.AreEqual(5, funnel2.FiredCount, "the once-law is session-scoped");
}
[Test]
public void T03_AdDualPositions_SubIdFamilies_WalkEndToEndOnRealPolicy()
{
    var sink = new CaptureSink();
    var telemetry = new TelemetryService();
    telemetry.AddSink(sink);
    var policy = new AdCapPolicy();
    // rescue walk: offer -&gt; show -&gt; complete -&gt; landed (R04 paradigm)
    var rescue = new G17RescueFunnel(telemetry, new MockAds(policy, () =&gt; DayOne), policy, () =&gt; DayOne);
    Assert.IsTrue(rescue.CanOffer(), "fresh day: the rescue category has budget");
    rescue.ReportOffer("chain-fail-left2");
    var task = rescue.Accept("chain-fail-left2");
    Assert.IsTrue(task.IsCompleted, "mock ads resolve synchronously");
    Assert.IsTrue(task.Result.Completed, "the rewarded view completes");
    rescue.ReportComplete("chain-fail-left2");
    rescue.ReportLanded("bubbles-cleared");
    // rescue capped walk (R05 paradigm): the drained category books capped
    // (the accept above already consumed one rescue slot, so drain to
    // exhaustion instead of assuming a fixed count)
    string rescueCat = AdCategories.CategoryOfPoint(G17RescueFunnel.PointId);
    int rescueDrain = 0;
    while (policy.TryConsumeRewarded(DayOne, rescueCat) &amp;&amp; rescueDrain++ &lt; 8) { }
    var rescueCapped = new G17RescueFunnel(telemetry, new MockAds(policy, () =&gt; DayOne), policy, () =&gt; Day
```

```
        Assert.IsFalse(rescueCapped.CanOffer(), "the rescue category is drained");
        rescueCapped.ReportCapped("daily-rescue-cap");
        // doubles walk: the same five-state family on the pre-reveal gate
        var doubles = new G17DoublesFunnel(telemetry, new MockAds(policy, () =&gt; DayOne), policy, () =&gt; DayOne)
        doubles.ReportOffer("reveal-rare");
        var dTask = doubles.Accept("reveal-rare");
        Assert.IsTrue(dTask.Result.Completed, "the doubles rewarded view completes");
        doubles.ReportComplete("reveal-rare");
        doubles.ReportLanded("second-copy");
        // drain the remaining double budget so the capped face is reachable
        string doubleCat = AdCategories.CategoryOfPoint(G17DoublesFunnel.PointId);
        int drainGuard = 0;
        while (policy.TryConsumeRewarded(DayOne, doubleCat) &amp;&amp; drainGuard++ &lt; 12) { }
        var doublesCapped = new G17DoublesFunnel(telemetry, new MockAds(policy, () =&gt; DayOne), policy, () =&gt; D
        Assert.IsFalse(doublesCapped.CanOffer(), "the doubles category is drained");
        doublesCapped.ReportCapped("daily-doubles-cap");
        Assert.AreEqual(new[]
        {
            UnboxEventIds.RescueOffer, UnboxEventIds.RescueShow,
            UnboxEventIds.RescueComplete, UnboxEventIds.RescueLanded,
            UnboxEventIds.RescueCapped,
            UnboxEventIds.DoubleOpenOffer, UnboxEventIds.DoubleOpenShow,
            UnboxEventIds.DoubleOpenComplete, UnboxEventIds.DoubleOpenLanded,
            UnboxEventIds.DoubleOpenCapped,
        }, sink.Ids.ToArray(),
            "both dual-position families walk offer-&gt;show-&gt;complete-&gt;landed plus capped end to end");
    }
    [Test]
    public void T04_SchemaContract_ZeroMissing_AllDeclaredIdsValidate()
    {
        Assert.IsTrue(G17TelemetrySchema.TryGetSchemaEventCount(out int count),
            "cfg_g17_events must parse and register");
        Assert.AreEqual(G17TelemetrySchema.SectionEventCount, count,
            "the schema registers the full section count (zero missing)");
        foreach (var id in G17TelemetrySchema.SectionIds())
        {
            Assert.IsTrue(G17TelemetrySchema.IdValidates(id),
                id + " must validate against the schema with zero warnings");
        }
        Assert.IsTrue(G17TelemetrySchema.RequiredFieldEnforced(),
            "required fields (step / elapsed_sec) must be enforced live");
        Assert.IsFalse(string.IsNullOrEmpty(G17TelemetrySchema.FunnelCategoryFor(G17RescueFunnel.PointId)),
            "the rescue seat carries the category field");
        Assert.IsFalse(string.IsNullOrEmpty(G17TelemetrySchema.FunnelCategoryFor(G17DoublesFunnel.PointId)),
            "the doubles seat carries the category field");
        Assert.IsTrue(G17TelemetrySchema.NewbieSeatTracksStepAndElapsed(),
            "the newbie seat carries step + elapsed_sec");
    }
    [Test]
    public void T05_RegistryBidirectional_SchemaNamesExactlyTheCodeVocabulary()
```

```
        {
            var declared = new HashSet&lt;string&gt;(G17TelemetrySchema.SectionIds());
            var codeVocabulary = new HashSet&lt;string&gt; { EventIds.NewbieStep };
            codeVocabulary.UnionWith(new[]
            {
                UnboxEventIds.RescueOffer, UnboxEventIds.RescueShow,
                UnboxEventIds.RescueComplete, UnboxEventIds.RescueLanded,
                UnboxEventIds.RescueCapped,
                UnboxEventIds.DoubleOpenOffer, UnboxEventIds.DoubleOpenShow,
                UnboxEventIds.DoubleOpenComplete, UnboxEventIds.DoubleOpenLanded,
                UnboxEventIds.DoubleOpenCapped,
            });
            CollectionAssert.AreEquivalent(codeVocabulary, declared,
                "the schema face names exactly the code-declared per-beat vocabulary");
            Assert.AreEqual(11, declared.Count, "eleven section events");
            // family roots stay schema anchors, never per-beat ids: the funnels
            // track the five sub-ids, the root constants name the families
            Assert.IsFalse(declared.Contains(UnboxEventIds.Rescue),
                "evt_rescue is the family anchor, not a per-beat id");
            Assert.IsFalse(declared.Contains(UnboxEventIds.DoubleOpen),
                "evt_double_open is the family anchor, not a per-beat id");
        }
    }
}
// G17UnboxPresenterTests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Core;
using Sy.Unbox;
using UnityEngine;
namespace Sy.Unbox.Tests
{
    /// &lt;summary&gt;
    /// S1-T003 slice a presenter gate (G12BoardPresenter parity paradigm):
    /// 1:1 command-to-event parity across the full craft chain, the seal
    /// click-fallback zero-threshold first op (H2 target), the pop pitch
    /// ladder feel anchor, deterministic slot anchors on the layout oracle,
    /// the HUD quartet readout face, and the rescue-seat view faces. Pure
    /// EditMode face - zero sim changes, zero scene wiring (slice b owns it).
    /// &lt;/summary&gt;
    [TestFixture]
    public class G17UnboxPresenterTests
    {
        static G17UnboxPresenter Make(int seed)
        {
            return new G17UnboxPresenter(new UnboxSim(seed));
        }
        static void AssertContainsKind(G17ViewCommand[] cmds, string kind)
        {
            for (int i = 0; i &lt; cmds.Length; i++)
```

```
        {
            if (cmds[i].Kind == kind) return;
        }
        Assert.Fail("expected command kind '" + kind + "' in drained batch");
    }
    [Test]
    public void V01_Open_Drains_1to1_Parity_With_Sim_Events()
    {
        var p = Make(11);
        var cmds = new List&lt;G17ViewCommand&gt;(p.Open());
        cmds.AddRange(p.Advance(0.1f));   // first_guide beat
        Assert.AreEqual(p.Sim.Events.Count, cmds.Count, "open+advance must drain 1:1 with sim events");
        Assert.AreEqual("caption", cmds[0].Kind);
        Assert.AreEqual("package", cmds[1].Kind);
        Assert.AreEqual("guide", cmds[2].Kind);
        Assert.AreEqual(p.Sim.Events[2].Time, cmds[2].TimeSec, 1e-6f, "beat must carry the sim second");
        Assert.AreEqual(1, p.PendingBoxCount, "one package on the table");
    }
    [Test]
    public void V02_FullChain_Every_Sim_Event_Maps_To_Exactly_One_Command()
    {
        var p = Make(23);
        var all = new List&lt;G17ViewCommand&gt;();
        all.AddRange(p.Open());
        all.AddRange(p.CutSealFromCanvas(G17UnboxLayout.SealCenter));
        int bubbles = p.Sim.CurrentPackage.Bubbles;
        for (int i = 0; i &lt; bubbles; i++)
        {
            all.AddRange(p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(i, bubbles)));
        }
        int latches = p.Sim.CurrentPackage.Latches;
        for (int i = 0; i &lt; latches; i++)
        {
            all.AddRange(p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(i, latches)));
        }
        all.AddRange(p.OpenLinerTap());
        all.AddRange(p.RevealTap(false));
        Assert.AreEqual(p.Sim.Events.Count, all.Count, "drained command total must equal sim event total (1:1 parity
        bool sawReveal = false, sawAlbum = false, sawShare = false;
        int revealTier = -1;
        for (int i = 0; i &lt; all.Count; i++)
        {
            if (all[i].Kind == "reveal") { sawReveal = true; revealTier = all[i].Tier; }
            if (all[i].Kind == "album") sawAlbum = true;
            if (all[i].Kind == "share") sawShare = true;
        }
        Assert.IsTrue(sawReveal, "reveal beat must drain");
        Assert.IsTrue(sawAlbum, "album beat must drain");
        Assert.IsTrue(sawShare, "share beat must drain");
        Assert.AreEqual(1, p.AlbumCollected, "first reveal collects one album item");
```

```
    Assert.AreEqual(24, p.AlbumTotal, "album badge denominator = frozen 24-slot table");
    Assert.AreEqual(G17UnboxPresenter.CoinAwardFor(revealTier), p.CoinsShown,
        "coin ledger = the reveal beat's own tier award (T004-B round-trip)");
    Assert.AreEqual(15, p.CoinsShown,
        "first box is forced Rare so the first reveal pays +15 (numeric pack sec6)");
    Assert.AreEqual(1, p.PendingBoxCount, "next package auto-spawns so the table never empties");
}
[Test]
public void V03_Seal_Tap_Fallback_ZeroThreshold_FirstOp()
{
    var p = Make(7);
    p.Open();
    // A plain tap far off the seal strip is still the same first
    // operation (click-fallback law, zero-threshold H2 target).
    var cmds = p.CutSealFromCanvas(new Vector2(0f, 0f));
    Assert.AreEqual(UnboxSim.Stage.Bubble, p.Sim.CurrentStage, "corner tap must cut the seal");
    Assert.AreEqual(1, cmds.Length);
    Assert.AreEqual("seal_cut", cmds[0].Kind);
    Assert.AreEqual(G17UnboxLayout.SealCenter, cmds[0].Canvas, "seal beat anchors the mouth band");
    Assert.IsTrue(G17UnboxLayout.OnSealStrip(G17UnboxLayout.SealCenter), "strip oracle must cover the mouth");
    Assert.IsFalse(G17UnboxLayout.OnSealStrip(new Vector2(0f, 0f)), "corner tap is off-strip (fallback law)");
}
[Test]
public void V04_Pop_Pitch_Ladder_1_00_Plus_0_02_i()
{
    var p = Make(5);
    p.Open();
    p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
    int bubbles = p.Sim.CurrentPackage.Bubbles;
    var pops = new List&lt;G17ViewCommand&gt;();
    for (int i = 0; i &lt; bubbles; i++)
    {
        pops.AddRange(p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(i, bubbles)));
    }
    int popCount = 0;
    for (int i = 0; i &lt; pops.Count; i++)
    {
        if (pops[i].Kind != "bubble_pop") continue;
        Assert.AreEqual(G17UnboxPresenter.PopPitchFor(popCount), pops[i].Pitch, 1e-6f,
            "pop " + popCount + " pitch must follow the 1.00 + 0.02 * i feel anchor");
        popCount++;
    }
    Assert.AreEqual(bubbles, popCount, "every bubble pops exactly once");
    Assert.AreEqual(1.00f, G17UnboxPresenter.PopPitchFor(0), 1e-6f);
    Assert.AreEqual(1.10f, G17UnboxPresenter.PopPitchFor(5), 1e-6f);
}
[Test]
public void V05_Bubble_Slot_Anchors_Unique_And_NearestFreeResolves()
{
    var p = Make(31);
```

```
        p.Open();
        p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
        int bubbles = p.Sim.CurrentPackage.Bubbles;
        var slots = new HashSet&lt;int&gt;();
        for (int i = 0; i &lt; bubbles - 1; i++)
        {
            var cmds = p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(i, bubbles));
            var beat = cmds[cmds.Length - 1];
            Assert.AreEqual("bubble_pop", beat.Kind);
            Assert.IsTrue(slots.Add(beat.Slot), "slot must be unique per pop");
            Assert.AreEqual(G17UnboxLayout.BubbleSlotCenter(beat.Slot, bubbles), beat.Canvas,
                "pop beat must carry its grid slot center");
        }
        // Far-corner feel tap: one bubble left, the nearest free slot resolves.
        var last = p.PopBubbleAt(new Vector2(-500f, -500f));
        var lastBeat = last[last.Length - 1];
        Assert.AreEqual("bubble_pop", lastBeat.Kind);
        Assert.GreaterOrEqual(lastBeat.Slot, 0);
        Assert.Less(lastBeat.Slot, bubbles);
        Assert.AreEqual(G17UnboxLayout.BubbleSlotCenter(lastBeat.Slot, bubbles), lastBeat.Canvas,
            "nearest free slot anchor must stay on the mesh");
        Assert.AreEqual(UnboxSim.Stage.Latch, p.Sim.CurrentStage, "last pop hands over to the latch rim");
    }
    [Test]
    public void V06_Rescue_Seat_Faces_Offer_Decline_Accept()
    {
        var p = Make(13);
        p.Sim.SetChainFailChance(1f);   // deterministic offer
        p.Open();
        p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
        while (p.Sim.CurrentPackage.Bubbles &gt; 2)
        {
            p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(0, p.Sim.CurrentPackage.Bubbles));
        }
        var fail = p.TryChainPopAt();
        AssertContainsKind(fail, "chain_fail");
        AssertContainsKind(fail, "rescue_offer");
        var decline = p.DeclineRescueTap();
        AssertContainsKind(decline, "rescue_decline");
        Assert.AreEqual(UnboxSim.Stage.Bubble, p.Sim.CurrentStage, "decline keeps hand-popping (zero penalty)");
        while (p.Sim.CurrentPackage.Bubbles &gt; 0)
        {
            p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(0, p.Sim.CurrentPackage.Bubbles));
        }
        Assert.AreEqual(UnboxSim.Stage.Latch, p.Sim.CurrentStage);
        // Finish the package so the next one spawns, then take the accept leg.
        while (p.Sim.CurrentPackage.Latches &gt; 0)
        {
            p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(0, p.Sim.CurrentPackage.Latches));
        }
```

```
        p.OpenLinerTap();
        p.RevealTap(false);
        p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
        while (p.Sim.CurrentPackage.Bubbles &gt; 2)
        {
            p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(0, p.Sim.CurrentPackage.Bubbles));
        }
        var fail2 = p.TryChainPopAt();
        AssertContainsKind(fail2, "rescue_offer");
        var accept = p.AcceptRescueTap();
        AssertContainsKind(accept, "rescue_accept");
        Assert.AreEqual(UnboxSim.Stage.Latch, p.Sim.CurrentStage, "accept clears the bubble layer in one beat");
        Assert.AreEqual(1, p.Sim.RescueCount);
    }
    [Test]
    public void V07_WrongStage_Op_Rejects_And_Baseline_Resyncs()
    {
        var p = Make(17);
        p.Open();
        var cmds = p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(0, 2));   // wrong stage (Seal)
        AssertContainsKind(cmds, "reject");
        Assert.AreEqual(UnboxSim.Stage.Seal, p.Sim.CurrentStage, "rejected op must not advance the stage");
        p.Sim.OneTapBaselineSteps();
        var baseline = p.Drain();
        AssertContainsKind(baseline, "baseline");
    }
    [Test]
    public void V08_Hud_Faces_And_Layout_Oracle()
    {
        var p = Make(29);
        Assert.AreEqual(0, p.CoinsShown, "coin honor face (S1 no-shop ruling)");
        Assert.AreEqual(24, p.AlbumTotal, "24-slot album table");
        p.Open();
        Assert.AreEqual(1, p.PendingBoxCount);
        var box = p.BoxOfCurrentPackage;
        Assert.NotNull(box, "first package (forced Rare) must resolve its box-table row");
        Assert.AreEqual("rare", box.rarity);
        Assert.AreEqual("A03", box.assetSlot, "rare-tier art slot (requirements doc sec4)");
        // Layout oracle pins: the 3-column mesh packs 8 -&gt; 3 rows, 15 -&gt; 5 rows.
        Assert.AreEqual(3, G17UnboxLayout.BubbleRows(8));
        Assert.AreEqual(5, G17UnboxLayout.BubbleRows(15));
        for (int i = 0; i &lt; 15; i++)
        {
            var c = G17UnboxLayout.BubbleSlotCenter(i, 15);
            Assert.GreaterOrEqual(c.x, G17UnboxLayout.PackageLeft);
            Assert.LessOrEqual(c.x, G17UnboxLayout.PackageRight);
            Assert.GreaterOrEqual(c.y, G17UnboxLayout.PackageBottom);
            Assert.LessOrEqual(c.y, G17UnboxLayout.PackageTop);
        }
        Assert.AreEqual(new Vector2(G17UnboxLayout.CoinX, G17UnboxLayout.HudY),
```

```
        G17UnboxLayout.HudAnchor(G17UnboxLayout.CoinX));
}
[Test]
public void V09_Reveal_Beat_Carries_FirstBox_ForcedRare_Tier()
{
    var p = Make(5);
    p.Open();
    p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
    int bubbles = p.Sim.CurrentPackage.Bubbles;
    for (int i = 0; i &lt; bubbles; i++) p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(i, bubbles));
    int latches = p.Sim.CurrentPackage.Latches;
    for (int i = 0; i &lt; latches; i++) p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(i, latches));
    p.OpenLinerTap();
    var cmds = p.RevealTap(false);
    G17ViewCommand reveal = null;
    for (int i = 0; i &lt; cmds.Length; i++) if (cmds[i].Kind == "reveal") reveal = cmds[i];
    Assert.IsNotNull(reveal, "reveal beat must drain");
    Assert.AreEqual(1, reveal.Tier, "first package is forced Rare (E4 H5 first-wow semantics)");
    Assert.AreEqual(2, G17UnboxPresenter.RevealToneCountFor(reveal.Tier), "Rare reveal = double-tone fanfare");
    Assert.IsFalse(G17UnboxPresenter.RevealHasPillarFor(reveal.Tier), "light pillar is the Hidden-tier signature
}
[Test]
public void V10_FourTier_Fanfare_Profiles_Are_Bot_Tellable()
{
    // tier ladder: single/double/triple/layered tones, pillar only on Hidden
    Assert.AreEqual(1, G17UnboxPresenter.RevealToneCountFor(0));
    Assert.AreEqual(2, G17UnboxPresenter.RevealToneCountFor(1));
    Assert.AreEqual(3, G17UnboxPresenter.RevealToneCountFor(2));
    Assert.AreEqual(4, G17UnboxPresenter.RevealToneCountFor(3));
    Assert.IsFalse(G17UnboxPresenter.RevealHasPillarFor(0));
    Assert.IsFalse(G17UnboxPresenter.RevealHasPillarFor(1));
    Assert.IsFalse(G17UnboxPresenter.RevealHasPillarFor(2));
    Assert.IsTrue(G17UnboxPresenter.RevealHasPillarFor(3), "gold light pillar = Hidden-tier signature");
}
[Test]
public void V11_Album_Stamp_Beat_Carries_Remaining_Count()
{
    var p = Make(31);
    p.Open();
    p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
    int bubbles = p.Sim.CurrentPackage.Bubbles;
    for (int i = 0; i &lt; bubbles; i++) p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(i, bubbles));
    int latches = p.Sim.CurrentPackage.Latches;
    for (int i = 0; i &lt; latches; i++) p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(i, latches));
    p.OpenLinerTap();
    var cmds = p.RevealTap(false);
    G17ViewCommand stamp = null;
    for (int i = 0; i &lt; cmds.Length; i++) if (cmds[i].Kind == "album") stamp = cmds[i];
    Assert.IsNotNull(stamp, "album beat must drain");
    Assert.AreEqual(1, p.AlbumCollected, "first reveal collects one item");
```

```
    Assert.AreEqual(p.AlbumTotal - p.AlbumCollected, stamp.Remaining, "stamp anchor = 24-slot table minus collec
    Assert.AreEqual(23, stamp.Remaining, "first reveal leaves 23 to go");
}
[Test]
public void V12_Coin_Award_Ladder_Matches_Numeric_Pack_Sec6()
{
    // honor income ladder: Common +5 / Rare +15 / Absurd +40 / Hidden +120
    Assert.AreEqual(5, G17UnboxPresenter.CoinAwardFor(0), "Common reveal pays +5");
    Assert.AreEqual(15, G17UnboxPresenter.CoinAwardFor(1), "Rare reveal pays +15");
    Assert.AreEqual(40, G17UnboxPresenter.CoinAwardFor(2), "Absurd reveal pays +40");
    Assert.AreEqual(120, G17UnboxPresenter.CoinAwardFor(3), "Hidden reveal pays +120");
    Assert.AreEqual(0, G17UnboxPresenter.CoinAwardFor(-1), "invalid tier pays nothing");
}
[Test]
public void V13_Coin_Ledger_Round_Trip_Across_Packages()
{
    var p = Make(37);
    p.Open();
    int expected = 0;
    int prev = 0;
    for (int pkg = 0; pkg &lt; 4; pkg++)
    {
        p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
        while (p.Sim.CurrentPackage.Bubbles &gt; 0)
        {
            p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(0, p.Sim.CurrentPackage.Bubbles));
        }
        while (p.Sim.CurrentPackage.Latches &gt; 0)
        {
            p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(0, p.Sim.CurrentPackage.Latches));
        }
        p.OpenLinerTap();
        var cmds = p.RevealTap(false);
        int tier = -1;
        for (int i = 0; i &lt; cmds.Length; i++) if (cmds[i].Kind == "reveal") tier = cmds[i].Tier;
        Assert.GreaterOrEqual(tier, 0, "every reveal beat must carry its tier");
        expected += G17UnboxPresenter.CoinAwardFor(tier);
        Assert.AreEqual(expected, p.CoinsShown,
            "ledger = sum of tier awards observed on the command stream (round-trip, package " + pkg + ")");
        Assert.Greater(p.CoinsShown, prev, "ledger strictly grows with every reveal");
        prev = p.CoinsShown;
    }
    Assert.GreaterOrEqual(p.CoinsShown, 15, "the forced-Rare first box seeds the ledger");
}
[Test]
public void V17_Double_Reveal_Books_Full_Plus_HalfPrice_Second_Item()
{
    var p = Make(71);
    p.Open();
    // Half-price ladder first (numeric pack sec6: +2/+7/+20/+60).
```

```
    Assert.AreEqual(2, G17UnboxPresenter.CoinHalfAwardFor(0), "Common second item pays +2");
    Assert.AreEqual(7, G17UnboxPresenter.CoinHalfAwardFor(1), "Rare second item pays +7");
    Assert.AreEqual(20, G17UnboxPresenter.CoinHalfAwardFor(2), "Absurd second item pays +20");
    Assert.AreEqual(60, G17UnboxPresenter.CoinHalfAwardFor(3), "Hidden second item pays +60");
    Assert.AreEqual(0, G17UnboxPresenter.CoinHalfAwardFor(-1), "invalid tier pays nothing");
    // Forced-Rare first box (tier 1): full +15 plus the half leg +7.
    p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
    while (p.Sim.CurrentPackage.Bubbles &gt; 0)
    {
        p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(0, p.Sim.CurrentPackage.Bubbles));
    }
    while (p.Sim.CurrentPackage.Latches &gt; 0)
    {
        p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(0, p.Sim.CurrentPackage.Latches));
    }
    p.OpenLinerTap();
    p.RequestDoubleOpenTap();
    Assert.AreEqual(1, p.Sim.CountEvents("double_open_request"), "the granted gate request books its beat");
    var cmds = p.RevealTap(true);
    int tier = -1;
    for (int i = 0; i &lt; cmds.Length; i++) if (cmds[i].Kind == "reveal") tier = cmds[i].Tier;
    Assert.GreaterOrEqual(tier, 0, "every reveal beat must carry its tier");
    Assert.AreEqual(G17UnboxPresenter.CoinAwardFor(tier) + G17UnboxPresenter.CoinHalfAwardFor(tier), p.CoinsShow
        "double reveal = full tier award + half-price second item (numeric pack sec6, S2-T005 leg)");
    Assert.AreEqual(22, p.CoinsShown, "forced-Rare first box double: 15 + 7 = 22");
}
// ---------------- S2-T004 slice b: save boot + persist wiring ----------------
/// &lt;summary&gt;In-memory ISaveService stub (check path only, zero platform coupling).&lt;/summary&gt;
sealed class MemorySave : ISaveService
{
    readonly Dictionary&lt;string, string&gt; _slots = new Dictionary&lt;string, string&gt;();
    public void Save(string slot, string payloadJson) { _slots[slot] = payloadJson; }
    public string Load(string slot)
    {
        string v;
        return _slots.TryGetValue(slot, out v) ? v : null;
    }
    public SyncResult CloudSync() { return new SyncResult(false, null); }
}
/// &lt;summary&gt;Walks one package from the seal all the way to the reveal
/// beat (the V13 loop extracted for the save-wiring tests).&lt;/summary&gt;
static void WalkOnePackageToReveal(G17UnboxPresenter p)
{
    p.CutSealFromCanvas(G17UnboxLayout.SealCenter);
    while (p.Sim.CurrentPackage.Bubbles &gt; 0)
    {
        p.PopBubbleAt(G17UnboxLayout.BubbleSlotCenter(0, p.Sim.CurrentPackage.Bubbles));
    }
    while (p.Sim.CurrentPackage.Latches &gt; 0)
    {
```

```
            p.PryLatchAt(G17UnboxLayout.LatchSlotCenter(0, p.Sim.CurrentPackage.Latches));
        }
        p.OpenLinerTap();
        p.RevealTap(false);
    }
    [Test]
    public void V14_BootFromSave_Restores_Every_Persisted_Face()
    {
        var save = new MemorySave();
        var seeded = new G17Profile
        {
            albumIds = new List&lt;string&gt; { "C01", "R01" },
            coins = 42,
            achievementsUnlocked = new List&lt;string&gt; { G17Achievements.FirstUnbox, G17Achievements.BoxClub },
            revealDayStreak = 3,
            longestRevealDayStreak = 5,
            lastRealDayUnix = 1700000000,
        };
        G17SaveSystem.Persist(save, seeded);
        var p = Make(61);
        long now = 1700000000 + 3600;   // later the same real day
        var status = p.BootFromSave(save, now, delegate { return now; });
        Assert.AreEqual(G17LoadStatus.Loaded, status, "versioned payload loads as-is");
        Assert.AreEqual(42, p.CoinsShown, "honor coin ledger seeds from the save");
        Assert.AreEqual(2, p.AlbumCollected, "album collection seeds into the sim truth");
        Assert.AreEqual(2, p.Achievements.UnlockedCount, "unlocked achievement set restores");
        Assert.IsTrue(p.Achievements.IsUnlocked(G17Achievements.BoxClub), "box_club restores");
        Assert.AreEqual(3, p.Sim.RevealDayStreak, "streak seeds through the boot policy (same-day keeps)");
        Assert.AreEqual(5, p.Sim.LongestRevealDayStreak, "longest restores as the ratchet maximum");
    }
    [Test]
    public void V15_Booted_Streak_Rides_Real_Days_SameDay_No_Stack_Next_Day_Extends()
    {
        var save = new MemorySave();
        var seeded = new G17Profile
        {
            revealDayStreak = 3,
            longestRevealDayStreak = 5,
            lastRealDayUnix = 1700000000,
        };
        G17SaveSystem.Persist(save, seeded);
        long daySecs = G17SaveSystem.RealDaySeconds;
        long now = 1700000000 + 3600;   // same real day as the saved anchor (gap = 0: 80000+3600 &lt; 86400)
        var p = Make(71);
        p.BootFromSave(save, now, delegate { return now; });
        p.Open();
        WalkOnePackageToReveal(p);
        Assert.AreEqual(3, p.Sim.RevealDayStreak, "a same-day reveal never stacks the chain");
        now += daySecs;   // the next real day
        WalkOnePackageToReveal(p);
```

```
            Assert.AreEqual(4, p.Sim.RevealDayStreak, "a next-day reveal extends the chain");
            Assert.AreEqual(5, p.Sim.LongestRevealDayStreak, "the best keeps the ratchet maximum");
        }
        [Test]
        public void V16_Persist_Beats_Round_Trip_And_Background_Never_Erases_A_Skipped_Day()
        {
            var save = new MemorySave();
            long daySecs = G17SaveSystem.RealDaySeconds;
            long day1 = 1700000000;
            // Day 1: a fresh game reveals its first box and the reveal beat persists.
            var p = Make(83);
            Assert.AreEqual(G17LoadStatus.NewGame, p.BootFromSave(save, day1, delegate { return day1; }));
            p.Open();
            WalkOnePackageToReveal(p);
            p.PersistAfterReveal(save, day1 + 60);
            Assert.AreEqual(1, p.Sim.RevealDayStreak, "the first reveal starts the chain");
            Assert.AreEqual(15, p.CoinsShown, "the forced-Rare first box pays +15");
            // Day 2: boot keeps the chain for the first reveal, but the player
            // reveals nothing and leaves to the background.
            long day2 = day1 + daySecs;
            var q = Make(84);
            Assert.AreEqual(G17LoadStatus.Loaded, q.BootFromSave(save, day2, delegate { return day2; }));
            Assert.AreEqual(1, q.Sim.RevealDayStreak, "next-day boot keeps the chain for the first reveal");
            Assert.AreEqual(15, q.CoinsShown, "the honor ledger round-trips through the boot");
            q.PersistOnBackground(save);
            // Day 3: the reveal-less day must have broken the chain - the
            // background persist must never stamp today as a reveal day.
            long day3 = day2 + daySecs;
            var r = Make(85);
            Assert.AreEqual(G17LoadStatus.Loaded, r.BootFromSave(save, day3, delegate { return day3; }));
            Assert.AreEqual(0, r.Sim.RevealDayStreak, "a day without a reveal breaks the seven-day chain");
            Assert.AreEqual(15, r.CoinsShown, "the ledger survives the skipped day untouched");
            Assert.AreEqual(1, r.AlbumCollected, "the album survives the skipped day untouched");
        }
    }
}
// UnboxGrayboxE4Tests.cs
using System;
using System.Collections.Generic;
using System.Diagnostics;
using NUnit.Framework;
using Sy.Unbox;
namespace Sy.Unbox.Tests
{
    /// &lt;summary&gt;
    /// E4 graybox gate for the UnboxIt candidate (ASMR unboxing x Chinese theme).
    /// 13 asserts in three sections per the M6 kit (m6_c_unbox_seed_eval_20260922.md):
    /// hooks 5/5 (red line 11 first-ten-seconds), bot 6/6 (200-package bare play),
    /// perf 2/2. Thresholds are design estimates recorded honestly (U099);
    /// construction values are documented per assert. Pure C#, EditMode only.
```

```
/// &lt;/summary&gt;
[TestFixture]
public class UnboxGrayboxE4Tests
{
    const float CraftStepFast = 0.08f;
    const float CraftStepNormal = 0.15f;
    const float CraftStepSlow = 0.25f;
    static bool IsCraftEvent(string name)
    {
        return name == "seal_cut" || name == "bubble_pop" || name == "latch_pry" ||
               name == "liner_open" || name == "reveal" || name == "chain_pop_success" ||
               name == "rescue_accept";
    }
    static int CountCraftBefore(UnboxSim sim, float deadline)
    {
        int n = 0;
        for (int i = 0; i &lt; sim.Events.Count; i++)
        {
            var e = sim.Events[i];
            if (e.Time &lt;= deadline &amp;&amp; IsCraftEvent(e.Name)) n++;
        }
        return n;
    }
    // ---------------- hooks 5/5 (red line 11, scripted first-ten-seconds) ----------------
    /// &lt;summary&gt;H1: caption at t=0 (no white-screen teaching). Construction:
    /// Open() emits caption at SimTime 0. Budget: ==0.00s.&lt;/summary&gt;
    [Test]
    public void H1_Caption_AtZero()
    {
        var sim = new UnboxSim(2026);
        sim.Open();
        float t = sim.FirstEventTime("caption");
        Assert.Less(Math.Abs(t), 1e-6f, "caption must fire at t=0, got " + t);
    }
    /// &lt;summary&gt;H2: first operation = swipe the seal. Construction: guide at
    /// 0.05s then CutSeal. Budget: &lt;=3s.&lt;/summary&gt;
    [Test]
    public void H2_FirstOp_SealCut_Within3s()
    {
        var sim = new UnboxSim(2026);
        sim.Open();
        sim.AdvanceClock(0.05f);
        bool ok = sim.CutSeal();
        Assert.IsTrue(ok, "first CutSeal must succeed");
        float t = sim.FirstEventTime("seal_cut");
        Assert.LessOrEqual(t, 3f, "first op at " + t + "s, budget 3s");
    }
    /// &lt;summary&gt;H3: first juicy feedback = first bubble pop. Construction: pop
    /// right after the seal cut. Budget: &lt;=1s.&lt;/summary&gt;
    [Test]
```

```
public void H3_FirstJuice_FirstBubblePop_Within1s()
{
    var sim = new UnboxSim(2026);
    sim.Open();
    sim.AdvanceClock(0.05f);
    sim.CutSeal();
    sim.AdvanceClock(0.05f);
    bool ok = sim.PopBubble();
    Assert.IsTrue(ok, "first PopBubble must succeed");
    float t = sim.FirstEventTime("bubble_pop");
    Assert.LessOrEqual(t, 1f, "first bubble pop at " + t + "s, budget 1s");
}
/// &lt;summary&gt;H4: at least 3 craft actions inside the first 10s (operation
/// density beat). Construction: scripted flow keeps popping bubbles.&lt;/summary&gt;
[Test]
public void H4_ThreeCraftActions_Within10s()
{
    var sim = new UnboxSim(2026);
    sim.Open();
    sim.AdvanceClock(0.05f);
    sim.CutSeal();
    for (int i = 0; i &lt; 3; i++)
    {
        sim.AdvanceClock(0.1f);
        sim.PopBubble();
    }
    int crafts = CountCraftBefore(sim, 10f);
    Assert.GreaterOrEqual(crafts, 3, "craft actions within 10s = " + crafts + ", need &gt;=3");
}
/// &lt;summary&gt;H5 (E3 cond 1): first wow = first rare reveal, and the multi-step
/// craft chain must complete within 3s so the difference vs competitor
/// one-tap is perceivable in the very first package. Construction: Open()
/// forces the first package Rare; full five-step chain scripted.&lt;/summary&gt;
[Test]
public void H5_FirstWow_RareReveal_MultiStepChain_Within3s()
{
    var sim = new UnboxSim(2026);
    sim.Open();
    float pkgStart = sim.FirstEventTime("package_start");
    sim.AdvanceClock(0.05f);
    sim.CutSeal();
    int stepCount = 1;
    while (sim.CurrentStage == UnboxSim.Stage.Bubble)
    {
        sim.AdvanceClock(0.1f);
        sim.PopBubble();
        stepCount++;
    }
    while (sim.CurrentStage == UnboxSim.Stage.Latch)
    {
```

```
            sim.AdvanceClock(0.1f);
            sim.PryLatch();
            stepCount++;
        }
        sim.AdvanceClock(0.05f);
        sim.OpenLiner();
        stepCount++;
        sim.AdvanceClock(0.05f);
        sim.Reveal(false);
        stepCount++;
        float revealT = sim.FirstEventTime("reveal");
        Assert.LessOrEqual(revealT - pkgStart, 3f, "first package chain took " + (revealT - pkgStart) + "s, budget 3
        // first wow = first reveal is rare (forced construction) so FirstEventTime(reveal) is the wow beat
        int oneTap = sim.OneTapBaselineSteps();
        Assert.AreEqual(1, oneTap, "competitor one-tap baseline must be 1 step");
        Assert.Greater(stepCount, oneTap, "multi-step chain (" + stepCount + ") must beat one-tap (1) - perceivable
    }
    // ---------------- bot 6/6 (bare play, fixed seed deterministic) ----------------
    static void PlayOnePackage(UnboxSim sim, Random rng, bool[] stylesSeen)
    {
        int roll = rng.Next(100);
        bool fast = roll &lt; 60;
        bool slow = roll &gt;= 90;
        stylesSeen[fast ? 0 : (slow ? 2 : 1)] = true;
        float step = fast ? CraftStepFast : (slow ? CraftStepSlow : CraftStepNormal);
        sim.AdvanceClock(step);
        sim.CutSeal();
        while (sim.CurrentStage == UnboxSim.Stage.Bubble)
        {
            if (sim.CurrentPackage.Bubbles &lt;= 2 &amp;&amp; !slow)
            {
                bool chained = sim.TryChainPop();
                if (!chained)
                {
                    if (sim.RescueOfferPending)
                    {
                        if (rng.Next(100) &lt; 80) sim.AcceptRescue();
                        else sim.DeclineRescue();
                    }
                    else
                    {
                        // S1-T005 production seat law (numeric pack sec3): the
                        // per-box rescue seat is spent, so the bot hand-pops the
                        // rest like the slow style. Bot construction adapted to
                        // the production law; the asserts are unchanged.
                        sim.AdvanceClock(step);
                        sim.PopBubble();
                    }
                }
                continue;
```

```
            }
            sim.AdvanceClock(step);
            sim.PopBubble();
        }
        while (sim.CurrentStage == UnboxSim.Stage.Latch)
        {
            sim.AdvanceClock(step);
            sim.PryLatch();
        }
        sim.AdvanceClock(step);
        sim.OpenLiner();
        bool doubleOpen = false;
        if (sim.CurrentPackage.Rarity &gt;= UnboxSim.Rarity.Rare)
        {
            sim.RequestDoubleOpen();
            doubleOpen = rng.Next(100) &lt; 80;
        }
        sim.Reveal(doubleOpen);
    }
    static UnboxSim PlayBareRun(int seed, int packages)
    {
        var sim = new UnboxSim(seed);
        sim.Open();
        var rng = new Random(seed);
        for (int i = 0; i &lt; packages; i++)
        {
            bool[] styles = new bool[3];
            PlayOnePackage(sim, rng, styles);
        }
        return sim;
    }
    /// &lt;summary&gt;B1: 200-package bare play (Random(42), mixed skills fast 60% /
    /// normal 30% / slow 10%). Asserts the full event surface fires.&lt;/summary&gt;
    [Test]
    public void B1_BarePlay_200Packages_AllEventSurface()
    {
        var sim = new UnboxSim(42);
        sim.Open();
        var rng = new Random(42);
        bool[] stylesSeen = new bool[3];
        for (int i = 0; i &lt; 200; i++) PlayOnePackage(sim, rng, stylesSeen);
        Assert.AreEqual(200, sim.RevealCount, "must reveal exactly 200 packages");
        Assert.AreEqual(201, sim.SpawnCount, "open spawn + 200 reveal spawns");
        Assert.Greater(sim.CountEvents("bubble_pop"), 0, "bubble pops must occur");
        Assert.Greater(sim.CountEvents("chain_pop_fail"), 0, "chain pop failures must occur");
        Assert.Greater(sim.CountEvents("rescue_accept"), 0, "rescue accepts must occur");
        Assert.Greater(sim.CountEvents("double_open_accept"), 0, "double opens must occur");
        Assert.AreEqual(200, sim.CountEvents("share_code"), "one share code per reveal");
        Assert.Greater(sim.AlbumLog.Count, 0, "album must collect");
        Assert.AreEqual(0, sim.CountEvents("op_rejected"), "bot plays in-order, zero rejections");
```

```
    Assert.IsTrue(stylesSeen[0] &amp;&amp; stylesSeen[1] &amp;&amp; stylesSeen[2], "all three bot styles must ap
}
/// &lt;summary&gt;B2: five-step chain health - every completed package segment is
/// strictly sequential (seal -&gt; bubbles -&gt; latches -&gt; liner -&gt; reveal), and
/// the multi-step chain depth dwarfs the one-tap baseline (E3 cond 1b).&lt;/summary&gt;
[Test]
public void B2_FiveStepChainHealth_StrictOrder_AndOneTapDiff()
{
    var sim = PlayBareRun(42, 200);
    int orderViolations = 0;
    // C345 attempt2: dual floors. All-segments floor 6 = designed rescue-compressed
    // chain (1 seal + 1 pop + 1 chain-clear + 1 latch + 1 liner + 1 reveal - the
    // bot always tries chain-pop at left&lt;=2, TryChainPop/ AcceptRescue clear the
    // bubble layer in ONE beat by design). Pure-manual floor 7 = original threshold
    // (1 seal + 3 bubbles + 1 latch + 1 liner + 1 reveal, arithmetic guarantee).
    int craftDepthMinAll = int.MaxValue;
    int craftDepthMinPure = int.MaxValue;
    int pureSegs = 0;
    int revealedSegs = 0;
    int segCrafts = 0, segReveal = 0, segPhase = 0;
    bool segStarted = false, segHasCraft = false, segHasRescue = false;
    for (int i = 0; i &lt; sim.Events.Count; i++)
    {
        var e = sim.Events[i];
        if (e.Name == "package_start")
        {
            if (segStarted &amp;&amp; segHasCraft &amp;&amp; segReveal == 0) orderViolations++;
            segStarted = true; segHasCraft = false; segHasRescue = false;
            segCrafts = 0; segReveal = 0; segPhase = 0;
            continue;
        }
        if (!segStarted) continue;
        if (e.Name == "seal_cut")
        {
            if (segPhase != 0) orderViolations++;
            segPhase = 1; segCrafts++; segHasCraft = true;
        }
        else if (e.Name == "bubble_pop")
        {
            if (segPhase != 1) orderViolations++;
            segCrafts++; segHasCraft = true;
        }
        else if (e.Name == "chain_pop_success" || e.Name == "rescue_accept")
        {
            if (segPhase != 1) orderViolations++;
            segPhase = 2; segCrafts++; segHasCraft = true; segHasRescue = true;
        }
        else if (e.Name == "latch_pry")
        {
            if (segPhase != 1 &amp;&amp; segPhase != 2) orderViolations++;
```

```
            segPhase = 2; segCrafts++; segHasCraft = true;
        }
        else if (e.Name == "liner_open")
        {
            if (segPhase != 2) orderViolations++;
            segPhase = 3; segCrafts++; segHasCraft = true;
        }
        else if (e.Name == "reveal")
        {
            if (segPhase != 3) orderViolations++;
            segReveal++; segCrafts++; revealedSegs++;
            if (segCrafts &lt; craftDepthMinAll) craftDepthMinAll = segCrafts;
            if (!segHasRescue)
            {
                if (segCrafts &lt; craftDepthMinPure) craftDepthMinPure = segCrafts;
                pureSegs++;
            }
            segStarted = false;
        }
    }
    Assert.AreEqual(200, revealedSegs, "all 200 packages must reveal");
    Assert.AreEqual(0, orderViolations, "strict sequential craft order violated");
    Assert.GreaterOrEqual(craftDepthMinAll, 6, "min craft depth " + craftDepthMinAll + " must be &gt;= 6 rescue-
    Assert.Greater(pureSegs, 0, "at least one pure manual segment must exist (decline path, ~7pct of 200 expecte
    Assert.GreaterOrEqual(craftDepthMinPure, 7, "pure-manual min craft depth " + craftDepthMinPure + " must be &
    int oneTap = sim.OneTapBaselineSteps();
    Assert.Greater(craftDepthMinAll, oneTap, "craft depth must beat one-tap baseline");
}
/// &lt;summary&gt;B3: album collection curve - monotonic non-decreasing, dedup
/// correct, and all four rarity tiers reached within 200 packages.&lt;/summary&gt;
[Test]
public void B3_AlbumCurve_Monotonic_Dedup_AllTiersReached()
{
    var sim = PlayBareRun(42, 200);
    var seen = new HashSet&lt;string&gt;();
    int dupes = 0;
    for (int i = 0; i &lt; sim.AlbumLog.Count; i++)
    {
        if (!seen.Add(sim.AlbumLog[i])) dupes++;
    }
    Assert.AreEqual(sim.AlbumIds.Count, seen.Count, "dedup must match the album set");
    Assert.AreEqual(sim.AlbumLog.Count, sim.CountEvents("album_add"), "album log must match album_add events");
    Assert.Greater(sim.AlbumLog.Count, 0, "album must not be empty");
    bool hasC = false, hasR = false, hasA = false, hasH = false;
    foreach (var id in sim.AlbumIds)
    {
        if (id.StartsWith("C")) hasC = true;
        else if (id.StartsWith("R")) hasR = true;
        else if (id.StartsWith("A")) hasA = true;
        else if (id.StartsWith("H")) hasH = true;
```

```
    }
    Assert.IsTrue(hasC &amp;&amp; hasR &amp;&amp; hasA &amp;&amp; hasH, "all four rarity tiers must be reached (
}
/// &lt;summary&gt;B4: rescue offer - forced chain failure raises the offer, accept
/// clears the bubble layer and restores the chain; decline keeps hand-popping
/// viable. Construction: SetChainFailChance(1.0) on a fresh sim.&lt;/summary&gt;
[Test]
public void B4_RescueOffer_Trigger_AcceptRestore_AndDeclinePath()
{
    var sim = new UnboxSim(123);
    sim.Open();
    sim.SetChainFailChance(1f);
    sim.AdvanceClock(0.1f);
    sim.CutSeal();
    while (sim.CurrentPackage.Bubbles &gt; 2) sim.PopBubble();
    bool chained = sim.TryChainPop();
    Assert.IsFalse(chained, "forced 100% fail must fail the chain pop");
    Assert.Greater(sim.CountEvents("chain_pop_fail"), 0, "chain_pop_fail must fire");
    Assert.IsTrue(sim.RescueOfferShown, "rescue offer must be shown");
    sim.AcceptRescue();
    Assert.AreEqual(0, sim.CurrentPackage.Bubbles, "rescue must clear the bubble layer");
    Assert.AreEqual(UnboxSim.Stage.Latch, sim.CurrentStage, "chain must restore to latch stage");
    Assert.AreEqual(1, sim.RescueCount, "rescue count must book");
    // decline path: finish current package then walk the next one down to two bubbles
    while (sim.CurrentStage == UnboxSim.Stage.Latch) sim.PryLatch();
    sim.OpenLiner();
    sim.Reveal(false);
    sim.CutSeal();
    while (sim.CurrentPackage.Bubbles &gt; 2) sim.PopBubble();
    bool chained2 = sim.TryChainPop();
    Assert.IsFalse(chained2, "still forced fail");
    sim.DeclineRescue();
    Assert.AreEqual(UnboxSim.Stage.Bubble, sim.CurrentStage, "decline keeps hand-popping");
    sim.PopBubble();
    sim.PopBubble();
    Assert.AreEqual(UnboxSim.Stage.Latch, sim.CurrentStage, "hand-popping must finish the layer");
    Assert.AreEqual(1, sim.RescueCount, "decline must not book a rescue");
}
/// &lt;summary&gt;B5: rewarded double open - rare+ package requests double open,
/// accepted reveal books two album copies; second request on the same
/// package is denied (once-per-package gate).&lt;/summary&gt;
[Test]
public void B5_DoubleOpen_Rewarded_OncePerPackageGate()
{
    var sim = new UnboxSim(77);
    sim.Open(); // first package forced Rare
    sim.AdvanceClock(0.1f);
    sim.CutSeal();
    while (sim.CurrentStage == UnboxSim.Stage.Bubble) sim.PopBubble();
    while (sim.CurrentStage == UnboxSim.Stage.Latch) sim.PryLatch();
```

```
    sim.OpenLiner();
    bool first = sim.RequestDoubleOpen();
    Assert.IsTrue(first, "first double open request must be accepted");
    bool second = sim.RequestDoubleOpen();
    Assert.IsFalse(second, "second request must be denied");
    Assert.AreEqual(1, sim.CountEvents("double_open_second_request_denied"), "denial event must book");
    int before = sim.AlbumLog.Count;
    sim.Reveal(true);
    Assert.AreEqual(2, sim.AlbumLog.Count - before, "accepted double open must add two copies");
    Assert.AreEqual(1, sim.DoubleOpenCount, "double open count must book");
}
/// &lt;summary&gt;B6: economy conservation + zero-gacha surface (E3 cond 3) -
/// album adds must equal the sum of reveal gains, every album_add shares
/// its reveal timestamp, and no gacha/lottery/purchase event exists.&lt;/summary&gt;
[Test]
public void B6_EconomyConservation_ZeroGachaSurface()
{
    var sim = PlayBareRun(42, 200);
    int gains = 0;
    float lastRevealTime = -1f;
    int albumAdds = 0;
    int misplaced = 0;
    for (int i = 0; i &lt; sim.Events.Count; i++)
    {
        var e = sim.Events[i];
        string n = e.Name;
        if (n == "gacha" || n == "lottery" || n == "paid_spin" || n == "purchase")
        {
            Assert.Fail("forbidden monetization event surfaced: " + n);
        }
        if (n == "reveal")
        {
            int gain = ParseGain(e.Detail);
            gains += gain;
            lastRevealTime = e.Time;
        }
        else if (n == "album_add")
        {
            albumAdds++;
            if (Math.Abs(e.Time - lastRevealTime) &gt; 1e-6f) misplaced++;
        }
    }
    Assert.AreEqual(gains, albumAdds, "album adds (" + albumAdds + ") must equal reveal gains (" + gains + ")");
    Assert.AreEqual(0, misplaced, "every album_add must share its reveal timestamp");
    Assert.AreEqual(albumAdds, sim.AlbumLog.Count, "log and events must agree");
}
static int ParseGain(string detail)
{
    int at = detail.IndexOf("gain=");
    if (at &lt; 0) return 0;
```

```
            int start = at + 5;
            int v = 0;
            while (start &lt; detail.Length &amp;&amp; detail[start] &gt;= '0' &amp;&amp; detail[start] &lt;= '9')
            {
                v = v * 10 + (detail[start] - '0');
                start++;
            }
            return v;
        }
        // ---------------- perf 2/2 (headless measured, budget per kit) ----------------
        /// &lt;summary&gt;P1: per craft-op cost across 20k full five-step packages
        /// (seal + pops + latches + liner + reveal). Budget: &lt;=0.05 ms/op
        /// (ArrowRush E4 family anchor 0.00470 ms/op).&lt;/summary&gt;
        [Test]
        public void P1_PerCraftOp_UnderBudget()
        {
            var sim = new UnboxSim(7);
            sim.Open();
            var sw = Stopwatch.StartNew();
            for (int i = 0; i &lt; 20000; i++)
            {
                sim.CutSeal();
                while (sim.CurrentStage == UnboxSim.Stage.Bubble) sim.PopBubble();
                while (sim.CurrentStage == UnboxSim.Stage.Latch) sim.PryLatch();
                sim.OpenLiner();
                sim.Reveal(false);
            }
            sw.Stop();
            double avgMs = sw.Elapsed.TotalMilliseconds / Math.Max(1, sim.OpCount);
            Console.WriteLine("[E4-P1] craft-op avgMs=" + avgMs.ToString("F6") + " ops=" + sim.OpCount + " packages=" +
            Assert.LessOrEqual(avgMs, 0.05, "craft op cost " + avgMs.ToString("F6") + " ms exceeds 0.05 budget");
        }
        /// &lt;summary&gt;P2: per-clock-step cost over 200k AdvanceClock ticks on a live
        /// mid-unbox board. Budget: &lt;=0.005 ms/tick (LineRescue family anchor).&lt;/summary&gt;
        [Test]
        public void P2_PerTick_UnderBudget()
        {
            var sim = new UnboxSim(9);
            sim.Open();
            sim.CutSeal();
            sim.PopBubble();
            var sw = Stopwatch.StartNew();
            for (int i = 0; i &lt; 200000; i++) sim.AdvanceClock(1f / 60f);
            sw.Stop();
            double avgMs = sw.Elapsed.TotalMilliseconds / 200000.0;
            Console.WriteLine("[E4-P2] tick avgMs=" + avgMs.ToString("F6") + " ticks=200000");
            Assert.LessOrEqual(avgMs, 0.005, "tick cost " + avgMs.ToString("F6") + " ms exceeds 0.005 budget");
        }
    }
}
```